

NUST CHIP DESIGN CENTRE

Physical Design

Lab Project
Backend of Single Cycle Processor

Name	***Sidra Naveed***
Instructor	*Ms. Hira Sohail*
Date	*15th October 2025*

Introduction to Synthesis:

Synthesis is the process of converting high level hardware description language written in RTL into gate level netlist. This netlist is made of standard cells which are provided through technology library. The objective of the synthesis to create a circuit which meets the functional correctness, timing, area and power requirements for the design.

In synthesis, the tools which we are using as Cadenece genus tool, maps RTL modules like registers counter and logic combinations onto logic gates, flip flops and multiplexers available in the liberty file.

Timing constraints, specified in the Synopsys Design Constraints (SDC) file, guide the tool to meet setup, hold, and propagation delay requirements.

Main Files:

Constraint File (.sdc)

The **constraint file**, which is called Synopsys Design Constraints (SDC) format. It provides the synthesis tool with the timing and design requirements it must follow. It defines important clock properties such as period, duty cycle, and rise/fall times. It also checks violations of setup and hold margins through clock uncertainty. It specifies how inputs and outputs should be timed by setting input delays, output delays, and input transition times relative to the clock.

By applying these constraints, the tool ensures the generated gate-level netlist behaves correctly under real hardware conditions, preventing setup/hold violations.

Script File (.tcl)

The **synthesis script**, written in Tcl language, automates the entire synthesis flow in tools like Cadence Genus. It starts by setting the design name, loading the standard cell libraries, and reading in the RTL files. Next, it applies the constraints from the .sdc file, elaborates the design, and runs the synthesis process—covering generic mapping, technology mapping, and optimization. At the end, it produces the mapped netlist, an SDF delay file, an updated constraint file, and detailed reports on timing, area, power, and gate count. Using a Tcl script guarantees repeatability, automation, and consistency, which are critical for efficient design iterations.

RTL File (.v)

The **RTL file**, written in Verilog, describes how the design works at the register-transfer level. It defines the behavior of modules like counters, state machines, or arithmetic circuits, and serves as the starting point for synthesis. The synthesis tool takes this high-level description and translates it into a gate-level netlist using cells from the target library. Since any bug in the RTL will carry forward into the synthesized hardware, the RTL must be thoroughly verified beforehand. In short, the Verilog RTL acts as the blueprint of the digital system being implemented.

Library File (.lib)

The **library file** contains the characterization data for the standard cells provided by a specific technology node (for example, 65 nm). Each .lib entry describes the behavior of a cell—its logic function, propagation delay, setup and hold times, power usage, and capacitance. Common cells include inverters, NAND gates, and flip-flops. During synthesis, the RTL design is mapped onto these cells while ensuring that all timing and power constraints are satisfied. Without this library, the synthesis tool cannot translate RTL into a real

hardware implementation. In this lab, the `tcbn65lptc.lib` library was used to map the counter design onto 65 nm standard cells.

Genus Tool:

Genus is the tool developed by cadence used to synthesis the rtl design. It helps till the frontend of the physical design.

Commands:

csh

This command starts the **C shell (Csh)**. Many EDA tools (like Cadence) are configured to run in csh because their setup scripts are written for it.

source cshrc

This command **loads the environment setup** from the `.cshrc` file. The following screenshot is taken from cshrc file given.

```
setenv CDS_LIC_FILE $LM_LICENSE_FILE
setenv CDS_LIC_QA_TesT "./qaLog.txt"
setenv LANG "en_US"
setenv QRC_EXTRACTION_ENABLE ""
setenv CDS_LIC_ONLY "1"
setenv CDS_AUTO_64BIT "ALL"
#setenv LD_PRELOAD "/usr/lib64/libssl.so.1.0.2k:/usr/lib64/libcrypto.so.1.0.2k"

setenv CDSHOME /home/cc/mnt/IC231
setenv INNOVUSHOME /home/cc/mnt/INNOVUS211/tools.lnx86

setenv XRUN_HOME /home/cc/mnt/XCELIUM2309

setenv QRC_HOME /home/cc/mnt/QUANTUS231

setenv SPECTRE_HOME /home/cc/mnt/SPECTRE231
```

Figure 1:cshrc file

This script prepares the environment by setting license details, tool paths, and runtime options, so that when you type genus -f script.tcl, the system immediately finds and runs the correct Cadence tool.

genus -f script.tcl

After system configuration, this command is used to start the cadence tool genus. It runs with the help of script.tcl. The file script.tcl works as the combination of different commands to run.

```
[cc@ncdc-0053 Sidra_PD]$ csh
[cc@ncdc-0053 Sidra_PD]$ source ~/cshrc
**************************************************
******        Welcome to Cadence Tools         *******
**************************************************

[cc@ncdc-0053 Sidra_PD]$ genus -f DFT_script.tcl
2025/10/01 15:29:57 WARNING This OS does not appear to be a Cadence supported Linux configuration.
2025/10/01 15:29:57 For more info, please run CheckSysConf in <cdsRoot/tools.lnx86/bin/checkSysConf <productId>
TMPDIR is being set to /tmp/genus_temp_22667_ncdc-0053_cc_tdn8GB
Cadence Genus(TM) Synthesis Solution.
Copyright 2023 Cadence Design Systems, Inc. All rights reserved worldwide.
Cadence and the Cadence logo are registered trademarks and Genus is a trademark
of Cadence Design Systems, Inc. in the United States and other countries.
```

Script.tcl:

```
# Print CPU information (useful for debug/log purposes)
if {[file exists /proc/cpuinfo]} {
    sh grep "model name" /proc/cpuinfo
    sh grep "cpu MHz"    /proc/cpuinfo
}

# Print hostname for log traceability
puts "Hostname : report timing [info hostname]"

# Setup library search path (standard cell libraries, etc.)
# IMPORTANT: Update the path to match your project environment
set_db / .init_lib_search_path /home/cc/HIRA_PD_cadence/COHORT3/Frontend/65_lp_libs

# Read the technology library file
read_libs {./65_lp_libs/tcbn65lptc.lib}


##############################################################################################################
## Design Setup
##############################################################################################################

# Define top-level design name (change this to your design top module)
set DESIGN       counter

# Set synthesis effort (generic synthesis step)
# Options: high | medium | low
set GEN_EFF      high

# Set mapping & optimization effort
# Options: high | medium | low
set MAP_OPT_EFF  high
```

Figure 2:script.tcl

Explanation of script.tcl:

The **script.tcl** file automates the entire synthesis flow in Cadence Genus. Here is the main flow of this script.

```
# Print CPU information (useful for debug/log purposes)
if {[file exists /proc/cpuinfo]} {
    sh grep "model name" /proc/cpuinfo
    sh grep "cpu MHz"    /proc/cpuinfo
}

# Print hostname for log traceability
puts "Hostname : report timing [info hostname]"

# Setup library search path (standard cell libraries, etc.)
# IMPORTANT: Update the path to match your project environment
set_db / .init_lib_search_path /home/cc/HIRA_PD_cadence/COHORT3/Frontend/65_lp_libs

# Read the technology library file
read_libs {./65_lp_libs/tcbn65lptc.lib}
```

It starts by setting up the environment—loading the technology libraries (.lib) and defining the top-level design module.

```
# Read HDL source files (Verilog 2001 in this case)
# IMPORTANT: Update the path if your Verilog source is located elsewhere
read_hdl -v2001 {/home/cc/HIRA_PD_cadence/COHORT3/Frontend/RTL/counter.v}

# Elaborate the top-level module
elaborate $DESIGN

# Check design for unresolved references (ensures all modules are linked correctly)
check_design -unresolved
```

Next, it reads in the RTL source file and elaborates it to catch any missing references.

```
# Read constraints file (SDC format)
# Contains clock definitions, input/output delays, etc.
read_sdc {/home/cc/HIRA_PD_cadence/COHORT3/Frontend/constraints/counter.sdc}
```

The design constraints from the .sdc file are then applied.

```
# Step 1: Generic Synthesis (technology-independent RTL → generic netlist)
set_db / .syn_generic_effort $GEN_EFF
syn_generic

# Step 2: Technology Mapping (map generic netlist to standard cells from library)
set_db / .syn_map_effort $MAP_OPT_EFF
syn_map

# Step 3: Netlist Optimization (timing, area, power optimizations)
set_db / .syn_opt_effort $MAP_OPT_EFF
syn_opt
```

Once the setup is complete, the script runs the synthesis in three stages: generic synthesis, technology mapping, and optimization for timing, power, and area.

```
# Write synthesized Verilog netlist (mapped to library cells)
write_hdl $DESIGN -mapped   >> ./Netlist/${DESIGN}_map.v

# Write constraints file for backend PnR (to be used in Innovus)
write_sdc $DESIGN           >> ./out/${DESIGN}_map.sdc

# Write delay annotation file (to be used in ModelSim/other simulators)
write_sdf -design $DESIGN   >> ./out/${DESIGN}_map.sdf

# Generate overall synthesis metrics in HTML format
report_metric -format html -file synthesis_report.html
```

After synthesis, it generates all the outputs: the mapped netlist (.v), an updated constraint file (.sdc), a delay annotation file (.sdf), and an HTML synthesis summary.

```
# Generate timing report (with full pin names for clarity)
report timing  -full_pin_names   >> ./RPT/${DESIGN}_timing.rpt

# Generate area report (cell count, area usage, etc.)
report area                      >> ./RPT/${DESIGN}_area.rpt

# Generate gates report (number of gates, breakdown of logic types)
report gates                     >> ./RPT/${DESIGN}_gates.rpt

# Generate power report (power estimation from synthesized netlist)
report power                     >> ./RPT/${DESIGN}_power.rpt

# Generate Quality of Results (QoR) report
# Includes logic levels, fanout, etc.
report qor -levels_of_logic      >> ./RPT/${DESIGN}_qor.rpt
```

It also produces detailed reports on timing, area, gate count, power, and overall quality of results (QoR).

Front End of Single Cycle Processor:

Changing Required in this script:

As this file was already provided with the paths defined, before running this file we had to change different paths according to our file location. First I changed the path of project environment file path, then the path of top.v and constraint.sdc. And after changing all the paths the script.tcl is run and following results are shown.

```
##############################################################################
## Environment Setup
##############################################################################

# Print CPU information (useful for debug/log purposes)
if {[file exists /proc/cpuinfo]} {
    sh grep "model name" /proc/cpuinfo
    sh grep "cpu MHz"    /proc/cpuinfo
}

# Print hostname for log traceability
puts "Hostname : report timing [info hostname]"

# Setup library search path (standard cell libraries, etc.)
# IMPORTANT: Update the path to match your project environment
set_db / .init_lib_search_path /home/cc/Downloads/Sidra_PD_UART/65_lp_libs

# Read the technology library file
read_libs {./65_lp_libs/tcbn65lptc.lib}


##############################################################################
## Design Setup
##############################################################################

# Define top-level design name (change this to your design top module)
set DESIGN       UART_Top

# Set synthesis effort (generic synthesis step)
# Options: high | medium | low
set GEN_EFF      high

# Set mapping & optimization effort
# Options: high | medium | low
set MAP_OPT_EFF  high
```

```
##############################################################################
## Read and Elaborate Design

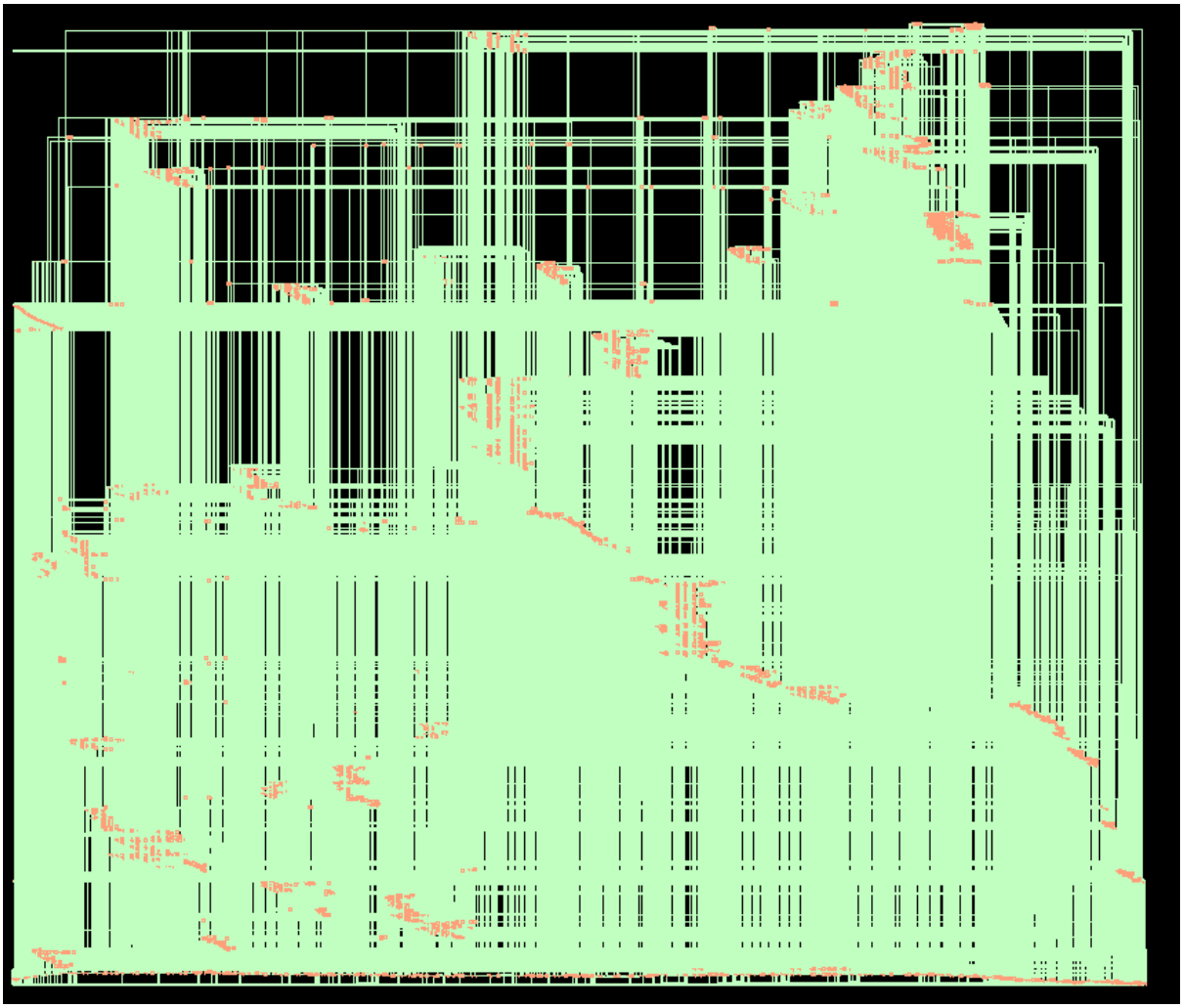
# IMPORTANT: Update the path if your Verilog source is located elsewhere

read_hdl -sv {/home/cc/Downloads/Sidra_PD_UART/RTL/UART_Top.sv}
read_hdl -sv {/home/cc/Downloads/Sidra_PD_UART/RTL/Transmitter.sv}
read_hdl -sv {/home/cc/Downloads/Sidra_PD_UART/RTL/Receiver.sv}
read_hdl -sv {/home/cc/Downloads/Sidra_PD_UART/RTL/baud_rate_generator.sv}
```

Explanation:

The script.tcl file is an automation script used to run the synthesis process for the processor design in the synthesis tool (such as Synopsys Design Compiler). It contains all the necessary commands to read the RTL files, define the working library, apply design constraints, and generate the synthesized gate-level netlist. The script usually starts by setting up the environment and specifying the target and link libraries, followed by reading the Processor Verilog files and the .sdc constraint file (which defines timing, clock, and input/output delays). After that, the script runs the synthesis command to map the RTL design onto standard cells, performs optimization for area and timing, and finally writes out reports for timing, area, and power, along with the synthesized netlist. This helps ensure a consistent, automated, and repeatable synthesis process without manually entering commands each time.

Synthesize:



Netlist:

This is netlist generated first time when we run the script.tcl, It shows how different gates, inputs and outputs are connected. It shows how different standard cells from library file are placed against RTL design. Different names are given like DFCN01, it specifies that it's a D flip flop. This netlist contains all the input and output connections, internal wires, and instantiated cells that make up the final synthesized design.

Files generated:

top_map.v(netlist)

top_gates.rpt

top_area.rpt

top_timing.rpt

top_qor.rpt

top_power.rpt

top_map.sdc (in “out” subdirectory)

top _map.sdf (in “out” subdirectory)

top_map.v:

top_map.v

home > cc > Downloads > LAB2B > Netlist > top_map.v

```
7   module top(clk, reset, instruction, read_data, result);
828   AOI221D0 g99112__4319(.A1 (n_3338), .A2 (n_2487), .B1 (n_2592), .B2
829        (\rf_registerfile[2] [22]), .C (n_3159), .ZN (n_3192));
830   AOI221D0 g99113__8428(.A1 (n_2487), .A2 (n_3337), .B1 (n_2592), .B2
831        (\rf_registerfile[2] [21]), .C (n_3158), .ZN (n_3191));
832   AO211D0 g99114__5526(.A1 (n_2592), .A2 (\rf_registerfile[2] [11]), .B
833        (n_3154), .C (n_3071), .Z (n_3190));
834   ND4D0 g99115__6783(.A1 (n_3174), .A2 (n_3041), .A3 (n_2775), .A4
835        (n_2776), .ZN (n_3189));
836   AOI221D0 g99116__3680(.A1 (n_2582), .A2 (\rf_registerfile[1] [0]),
837        .B1 (n_2594), .B2 (\rf_registerfile[15] [0]), .C (n_3137), .ZN
838        (n_3188));
839   AOI221D0 g99117__1617(.A1 (n_2582), .A2 (\rf_registerfile[1] [20]),
840        .B1 (n_2594), .B2 (\rf_registerfile[15] [20]), .C (n_3148), .ZN
841        (n_3187));
842   AOI221D0 g99118__2802(.A1 (n_2592), .A2 (\rf_registerfile[2] [19]),
843        .B1 (n_2582), .B2 (\rf_registerfile[1] [19]), .C (n_3150), .ZN
844        (n_3186));
845   AOI221D0 g99119__1705(.A1 (n_2582), .A2 (\rf_registerfile[1] [18]),
846        .B1 (n_2594), .B2 (\rf_registerfile[15] [18]), .C (n_3146), .ZN
847        (n_3185));
848   AOI221D0 g99120__5122(.A1 (n_2582), .A2 (\rf_registerfile[1] [17]),
849        .B1 (n_2594), .B2 (\rf_registerfile[15] [17]), .C (n_3145), .ZN
850        (n_3184));
851   AOI221D0 g99121__8246(.A1 (n_2592), .A2 (\rf_registerfile[2] [16]),
852        .B1 (n_2594), .B2 (\rf_registerfile[15] [16]), .C (n_3138), .ZN
853        (n_3183));
854   AOI221D0 g99122__7098(.A1 (n_2592), .A2 (\rf_registerfile[2] [15]),
855        .B1 (n_2582), .B2 (\rf_registerfile[1] [15]), .C (n_3155), .ZN
856        (n_3182));
857   AOI221D0 g99123__6131(.A1 (n_2592), .A2 (\rf_registerfile[2] [13]),
858        .B1 (n_2594), .B2 (\rf_registerfile[15] [13]), .C (n_3143), .ZN
```

Explanation:

This part of the synthesized top_map.v file shows the gate-level version of my single-cycle processor frontend after synthesis. It means that my Verilog code has been converted into basic hardware components like AND, OR, NAND, NOR, and NOT gates. Each line represents how these gates are connected to form the logic of the processor, such as the register file and instruction decoding parts. Signals like n_2019 and \rf_registerfile[15][11] are internal connections and register bits used by the design. In simple terms, this file shows that my processor's RTL design has been successfully converted into actual hardware logic that can be implemented on a chip.

Top gates Report:

```
NR2XD3            2     12.240    tcbn65lptc
NR3D0             4      8.640    tcbn65lptc
NR4D0             5     12.600    tcbn65lptc
OA211D0           1      2.880    tcbn65lptc
OA211D1           1      2.880    tcbn65lptc
OA21D0            9     22.680    tcbn65lptc
OA21D1           14     35.280    tcbn65lptc
OA221D0           4     14.400    tcbn65lptc
OA22D0            2      5.760    tcbn65lptc
OAI211D0         12     30.240    tcbn65lptc
OAI21D0          33     71.280    tcbn65lptc
OAI21D1          34     73.440    tcbn65lptc
OAI221D0         33     95.040    tcbn65lptc
OAI222D0          9     29.160    tcbn65lptc
OAI22D0          42    105.840    tcbn65lptc
OAI22D1           1      2.520    tcbn65lptc
OAI31D0           4     10.080    tcbn65lptc
OAI32D0           2      5.760    tcbn65lptc
OR2D0            11     23.760    tcbn65lptc
OR2XD1            8     17.280    tcbn65lptc
OR3D1             1      2.520    tcbn65lptc
OR4D0             7     20.160    tcbn65lptc
XNR2D1            3     10.800    tcbn65lptc
XOR3D1            1      6.120    tcbn65lptc
-----------------------------------------------
total          4759  16794.720
```

```
Type       Instances    Area   Area %
----------------------------------------
sequential          1057  7182.000   42.8
inverter             147   173.160    1.0
buffer                32    46.080    0.3
logic               3523  9393.480   55.9
physical_cells         0     0.000    0.0
----------------------------------------
total               4759 16794.720  100.0
```

Explanation:

This gate report shows the detailed result of the synthesis process for my single-cycle processor frontend. It lists all the logic gates and flip-flops that the synthesis tool (Cadence Genus) used to convert my Verilog code into actual hardware. Each type of gate, such as AND, OR, NAND, NOR, AOI (AND-OR-Invert), and OAI (OR-AND-Invert), is shown along with how many times it is used and how much area it occupies on the chip. The “sequential” part includes flip-flops and registers, which store data between clock cycles. The report also

shows that there are a total of 4614 cells used, taking a total chip area of about 16593 µm². Around 43% of the area is used by sequential elements (registers and flip-flops), and about 56% is used by logic gates that perform operations. In simple words, this report tells how my processor design has been physically built from basic digital building blocks and how much area each part of it occupies on the chip.

Top_area.rpt:

```
====================================================================
  Generated by:           Genus(TM) Synthesis Solution 21.18-s082_1
  Generated on:           Oct 05 2025  05:59:30 pm
  Module:                 top
  Operating conditions:   NCCOM (balanced_tree)
  Wireload mode:          segmented
  Area mode:              timing library
====================================================================

Instance Module  Cell Count  Cell Area  Net Area   Total Area      Wireload
--------------------------------------------------------------------------------
top                   4759  16794.720     0.000    16794.720 ZeroWireload (S)
  (S) = wireload was automatically selected
```

Explanation:

This area report shows the total hardware size of my single-cycle processor frontend after synthesis. It tells that the design, named “top”, contains 4759 cells (logic gates and flip-flops) in total. The cell area is 16794.720 µm², which represents the total silicon area required to implement all the components. The net area is 0.000, meaning the wiring area was not separately calculated because a “zero wireload” model was used — this means the tool assumed ideal wire connections for simplicity. In short, this report confirms that my processor design is compact and occupies a total chip area of 16794.720 µm², based only on logic cell usage.

top_timing.rpt:

```
Path 1: MET (5028 ps) Late External Delay Assertion at pin result[31]
           Group: clk
     Startpoint: (R) pc_pc_out_reg[3]/CP
           Clock: (R) clk
        Endpoint: (F) result[31]
           Clock: (R) clk

                          Capture       Launch
            Clock Edge:+    10000            0
            Src Latency:+       0            0
            Net Latency:+       0 (I)        0 (I)
                Arrival:=   10000            0

           Output Delay:-    1000
           Uncertainty:-     2000
     Required Time:=         7000
           Launch Clock:-       0
              Data Path:-    1972
                  Slack:=    5028
```

Explanation:
This timing report shows the setup timing check for the single-cycle processor frontend. The analysis is for the path that starts from the instruction input and ends at the result output. The total data path delay is 200 ps, and after considering the input delay (1000 ps), output delay (1000 ps), and uncertainty (2000 ps), the required time is 7000 ps.
Since the slack is 5028 ps, We didn’t modified it, because after all the synthesis, it won’t get negative.

top_qor.rpt:

```
Clock Period
-------------
clk   4200.0


  Cost    Critical         No of gates on  Violating
 Group   Path Slack  TNS    Critical Path    Paths
-----------------------------------------------------
clk             0.0   0.0              6          0
default    No paths   0.0
-----------------------------------------------------
Total                 0.0                         0

Instance Count
--------------
Leaf Instance Count             4759
Physical Instance count            0
Sequential Instance Count       1057
Combinational Instance Count    3702
Hierarchical Instance Count        0

Area
----
Cell Area                          16794.720
Physical Cell Area                 0.000
Total Cell Area (Cell+Physical)    16794.720
Net Area                           0.000
Total Area (Cell+Physical+Net)     16794.720

Max Fanout                         1024 (clk)
Min Fanout                         1 (ALUResult[0])
Average Fanout                     3.1
Terms to net ratio                 4.1281
Terms to instance ratio            4.2383
Runtime                            58.69125799999999 seconds
Elapsed Runtime                    58 seconds
Genus peak memory usage            1960.52
Innovus peak memory usage          no_value
Hostname                           ncdc-0053
```

Explanation:

This report summarizes how the synthesized single cycle performed. The tool used a clock period of 4200 ps and the design has a positive slack of 0 ps on the critical clock path — so it meets timing with a small margin. The critical path spans 12 gates, and there are no timing violations (TNS = 0). The netlist contains 4759 leaf instances (1057 sequential registers and 3702 combinational cells) and a total cell area of 16794.720 µm². The

clock net has the highest fanout (1024) while the average fanout is 3.1, which indicates reasonable net loading overall.

top_power.rpt:

```
Instance: /top
Power Unit: W
PDB Frames: /stim#0/frame#0
  ------------------------------------------------------------------------
  Category         Leakage     Internal    Switching        Total    Row%
  ------------------------------------------------------------------------
    memory     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  register     3.28131e-07  2.74416e-03  1.17825e-04  2.86231e-03  75.35%
     latch     5.81704e-09  2.38083e-05  5.54467e-06  2.93588e-05   0.77%
     logic     2.94966e-07  5.30902e-04  3.75585e-04  9.06781e-04  23.87%
      bbox     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
     clock     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
       pad     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        pm     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  ------------------------------------------------------------------------
  Subtotal     6.28915e-07  3.29887e-03  4.98954e-04  3.79845e-03  99.99%
Percentage           0.02%       86.85%       13.14%      100.00% 100.00%
  ------------------------------------------------------------------------
```

This power table is for the /top module and shows how much power each part of the design uses. The total power is ≈1.564 mW (1.56441×10^{-3} W). That breaks down into leakage (very small, 0.000000619 W), internal power (≈1.368 mW, 87.5%) and switching power (≈0.196 mW, 12.5%). Most of the power (about 76.4%) is consumed by registers (flip-flops and storage) because they toggle and charge internal nodes during clocked operation. Combinational logic uses about 22.8%, and latches are negligible (~0.8%). The clock, pads and memories show no separate power here. In simple terms: the design's dynamic activity (mainly inside registers) is the main source of power — total is low (~1.6 mW) and dominated by internal register activity.

top_map.sdc(out directory):

```
set sdc_version 2.0

set_units -capacitance 1000fF
set_units -time 1000ps

# Set the current design
current_design top

create_clock -name "clk" -period 4.2 -waveform {0.0 2.1} [get_ports clk]
set_clock_transition 0.1 [get_clocks clk]
set_clock_gating_check -setup 0.0
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports reset]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {instruction[31]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {instruction[30]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {instruction[29]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {instruction[28]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {instruction[27]}]
```

```
set_output_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {result[0]}]
set_wire_load_mode "segmented"
set_clock_uncertainty -setup 2.0 [get_clocks clk]
set_clock_uncertainty -hold 1.0 [get_clocks clk]
set_clock_uncertainty -setup 0.01 [get_ports clk]
set_clock_uncertainty -hold 0.01 [get_ports clk]
```

This file is generated by cadence tool genus. It has the timing and design constraints that define how the synthesized design should behave in terms of clock, input and output. It tells how much margin should be used in timing analysis. It defines a 10ns clock period for the clock with 0.1ns of rise/fall transition and sets input and output delay constraints of 1ns for reset and count signals.

top_map.sdf:

```
  (CELL
     (CELLTYPE "INVD1")
     (INSTANCE g41878)
     (DELAY
        (ABSOLUTE
          (PORT I (::0.0))
          (IOPATH I ZN (::278) (::251))
        )
     )
  )
  (CELL
     (CELLTYPE "CKND2D1")
     (INSTANCE g98997__2398)
     (DELAY
        (ABSOLUTE
          (PORT A1 (::0.0))
          (PORT A2 (::0.0))
          (IOPATH A1 ZN (::154) (::144))
          (IOPATH A2 ZN (::155) (::133))
        )
     )
  )
```

Gate Level Simulation (Post Synthesis):

Gate level simulation is done after synthesis to check that circuit will work once it is converted from RTL into logic gates. It helps us to verify that the functionality hasn't changed and that delays from real gates and wires don't cause issues.

Files needed for gate level Simulation:

1. **Synthesized netlist:**

It is the top_map.v file which is Verilog code after synthesis. In this file design is converted into actual standard cells.

2. **Testbench file:**

This is tb_top.v file it is the testbench same as we used before synthesis to apply inputs and output checks.

3. **SDF file:**

This is top_map.sdf file whoch contains actual delay values of every gate and wire. It is generated by genus.

4. **Technology library file:**

This file is tcbn651lp.v, and it has the Verilog modules of the standard cells used in the netlist.

Changing SDF Annotation:

This tells the simulator to use the real gate delays from synthesis.

Running the Simulation:

Csh
It opens the C shell.
Source cshrc
It sets up environment variable and tools.

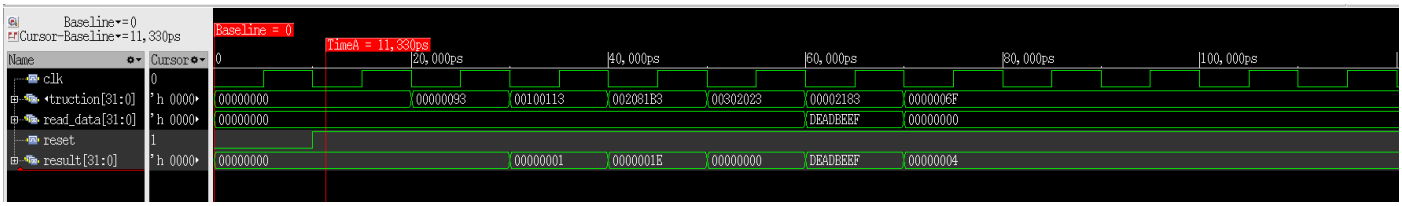
xrun -timescale 1ns/10ps top_map.v tb_top.sv -v tcbn65lp.v -access +rwc -define SDF_TEST -mess -gui

Design summary:

```
  Design hierarchy summary:
                            Instances  Unique
          Modules:               4616      91
          UDPs:                  5347       4
          Primitives:           15247       9
          Timing outputs:        4676      57
          Registers:             1062      12
          Scalar wires:          5736       -
          Expanded wires:          96       3
          Initial blocks:           3       3
          Pseudo assignments:       5       -
          Timing checks:        10983       -
          Interconnect:         15176       -
          Simulation timescale:   10ps
  Writing initial simulation snapshot: worklib.top_tb:sv
```

Explanation:
This design hierarchy symmary is shown after running GLS. It shows the structural breakdown of my simulated design. It tells how many modules, primitives and componenets it contains after synthesis.

Cadence Waveform:



Waveform:

Explanation:

In my waveform, At the start, the clock keeps toggling every 5 ns, and reset is applied briefly to initialize all signals. After reset, the test begins by sending a sequence of RISC-V instructions to the processor. Each instruction (like addi, add, sw, lw, and jal) executes in a new cycle, and the waveform shows changes in instruction, read_data, and result signals accordingly.

Backend:

Floor planning is the initial step in physical design where you define the approximate locations of major components (blocks) within the chip. This includes the placement of macros, standard cells, and other important elements. The goal of floor planning is to meet design constraints such as area utilization, timing, and routability. It provides a high-level view of how the different components of the design will be organized on the chip.

After floor planning, the step of pin assignment according to the design is performed.

Power planning involves designing the power distribution network to ensure that each part of the chip receives the required power. This includes creating power grids, distributing power and ground connections, and minimizing voltage drops. Placement: Assign locations for each logic cell and other components on the chip. Optimize placement for factors like wirelength, timing, and congestion.

Clock Tree Synthesis (CTS): Build a clock distribution network to provide a stable and synchronized clock to all sequential elements. Optimize for skew, balancing, and power.

Routing: Connect the pins of logic cells and other components using metal layers. Resolve congestion and optimize for wirelength, parasitics, and timing.

Optimization: Perform various optimization techniques to improve the design metrics. Timing optimization to meet the required performance. Power optimization to reduce overall power consumption.

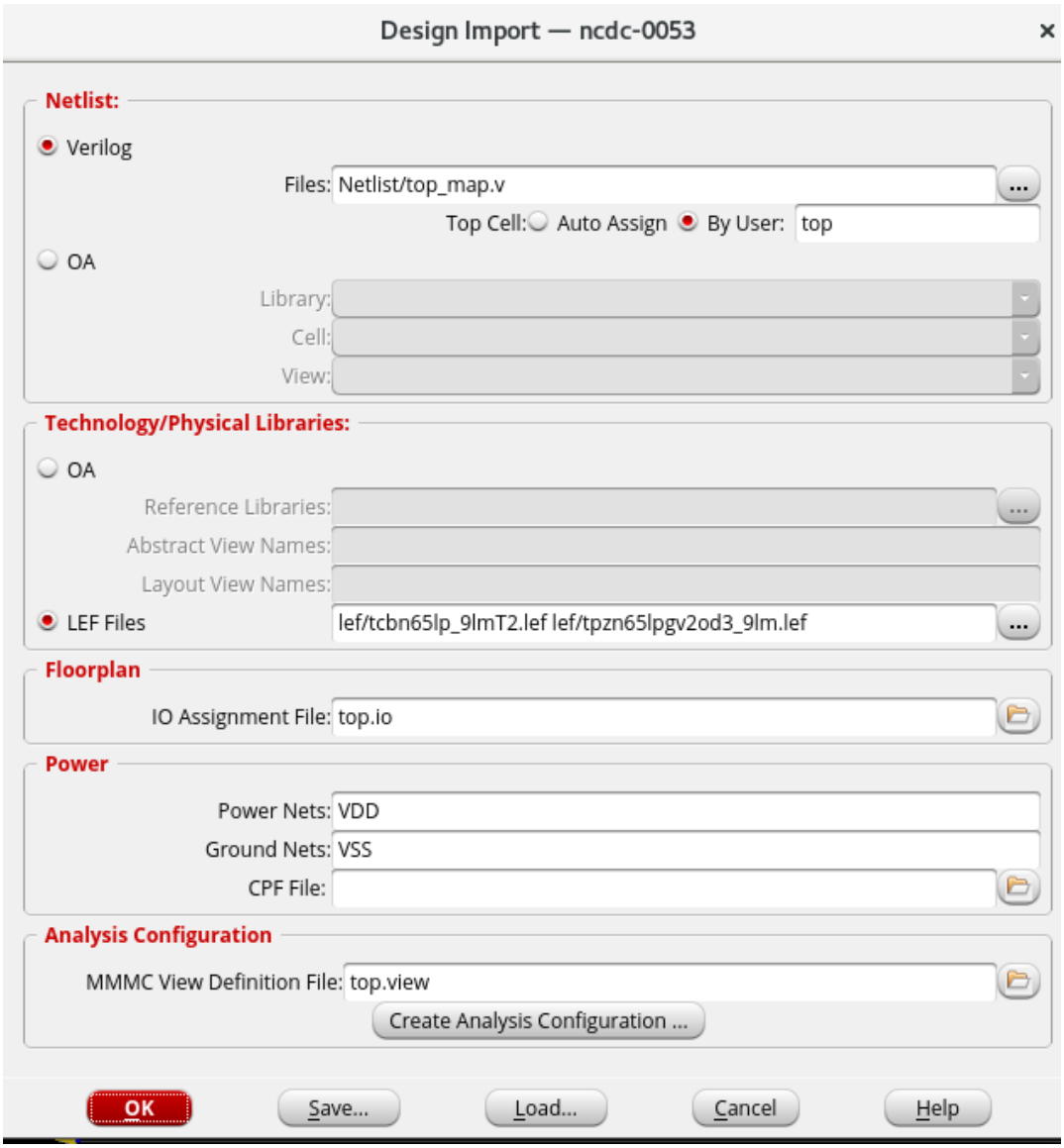
Static Timing Analysis (STA): Analyze the timing characteristics of the design to ensure that it meets the required performance. Identify and fix timing violations.

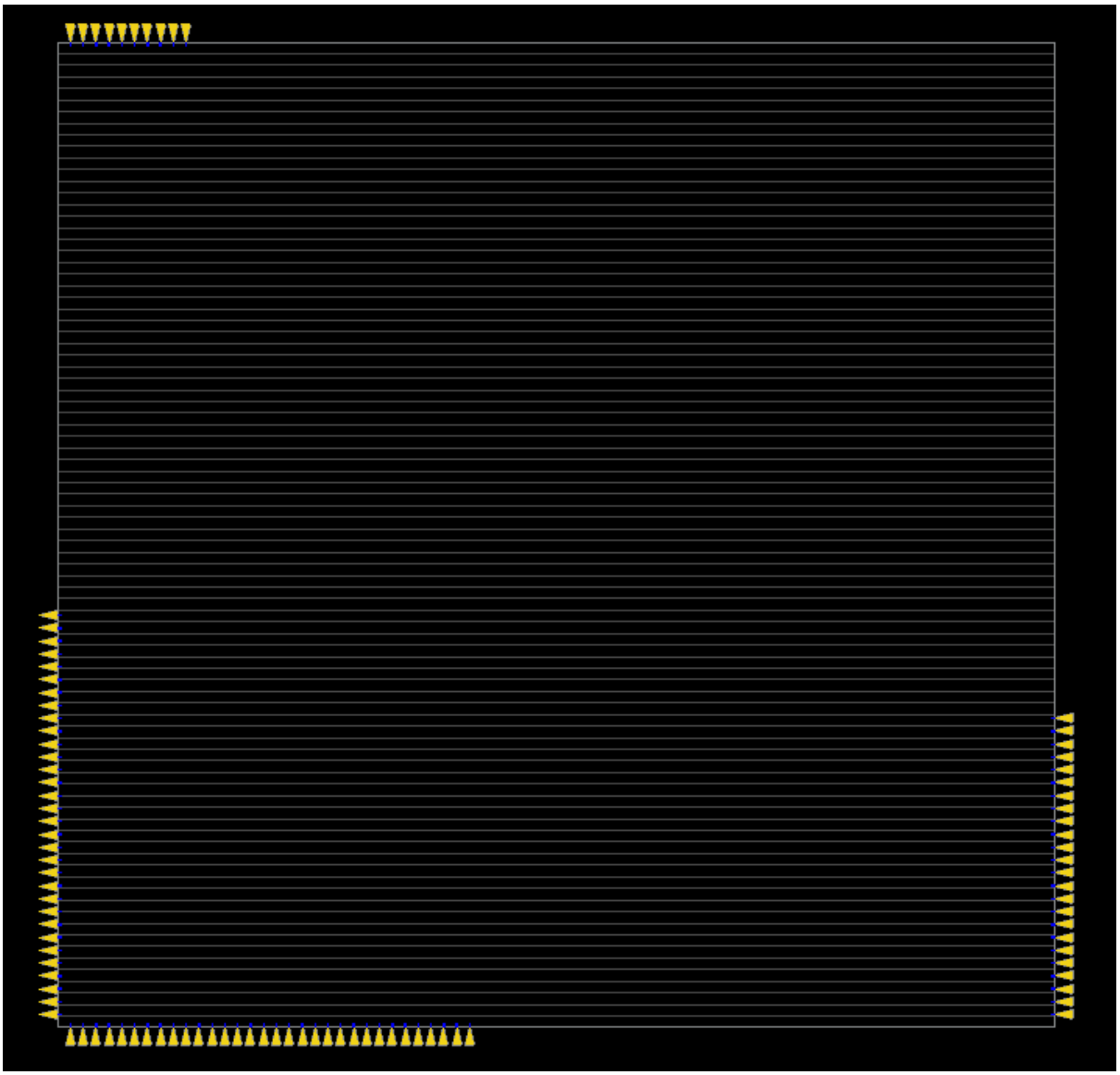
Commands:

csh

source cshrc

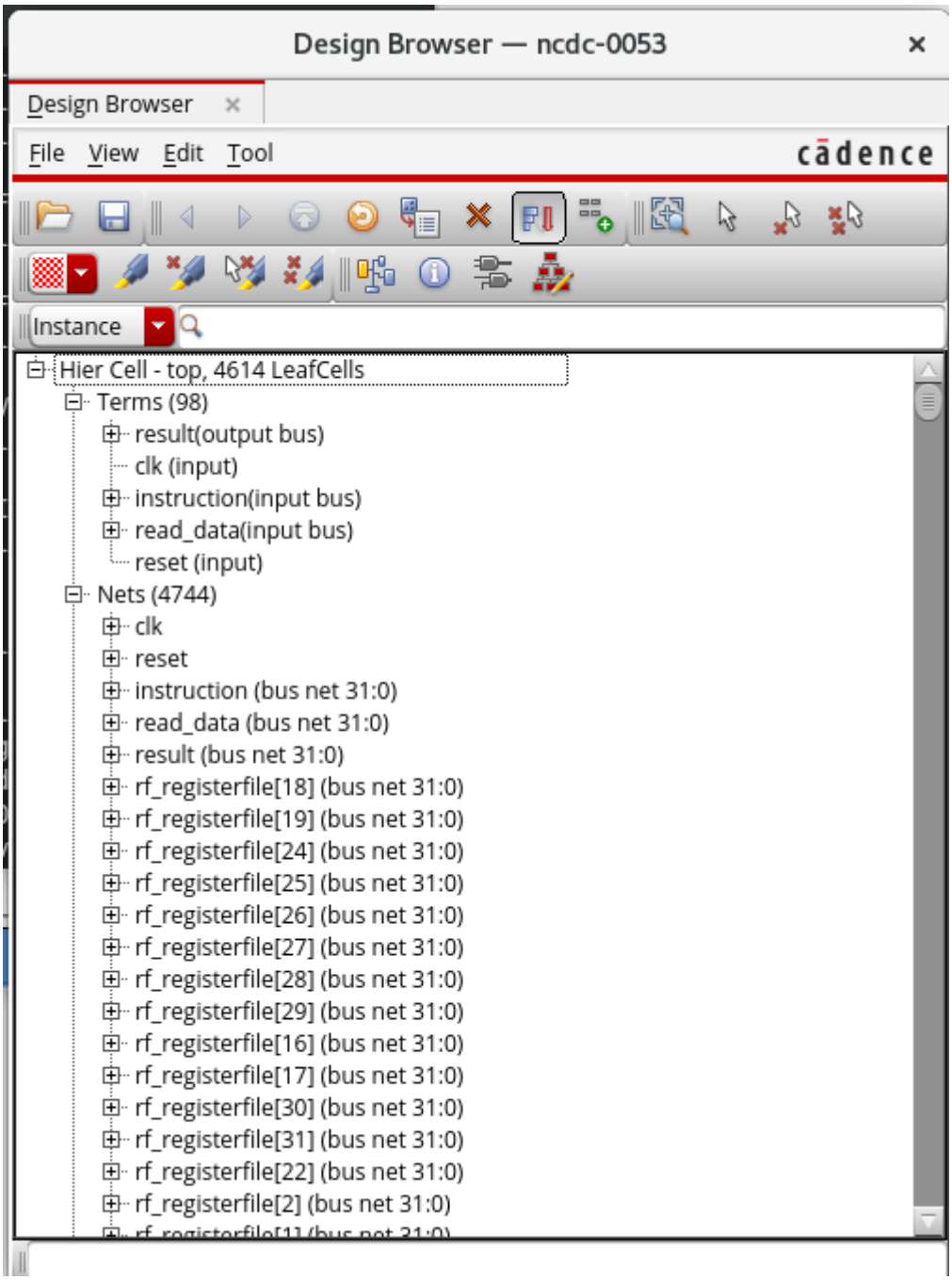
innovus





Design Browser:

Viewing the Design Hierarchy:I Go to tools – Design browser.

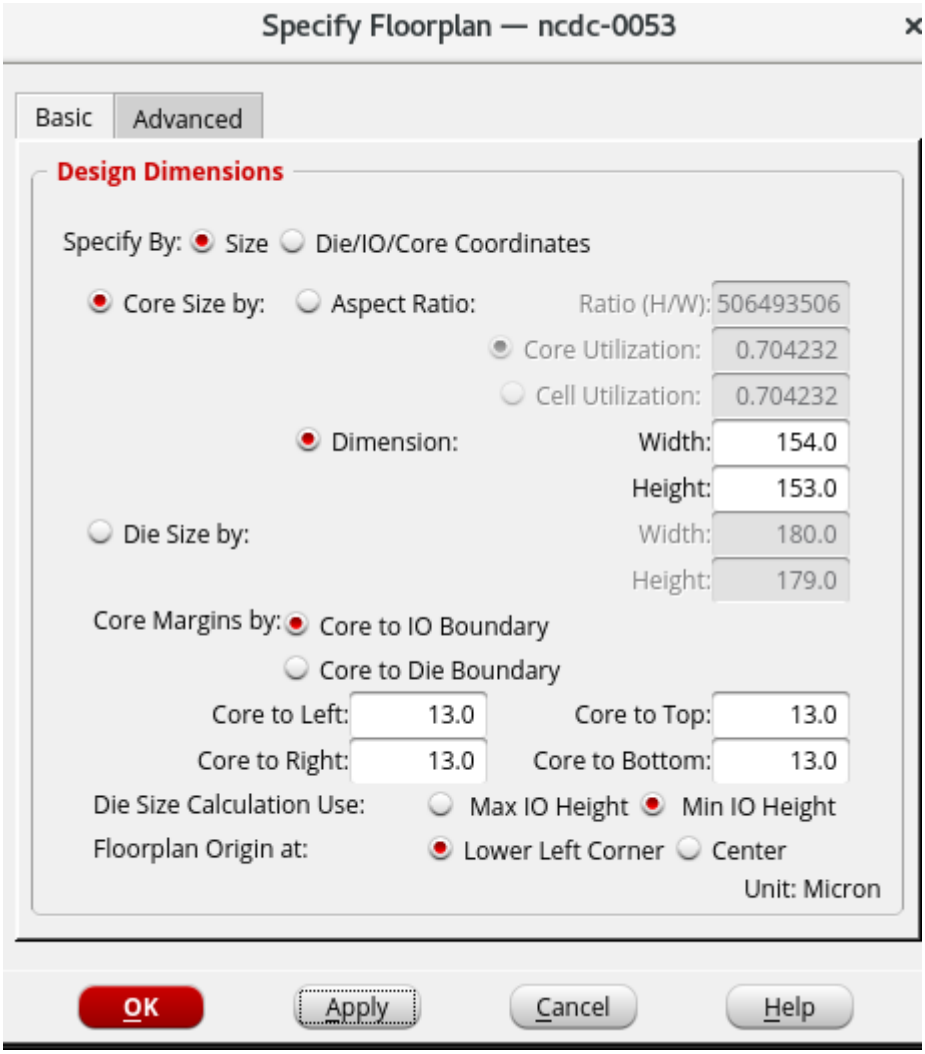


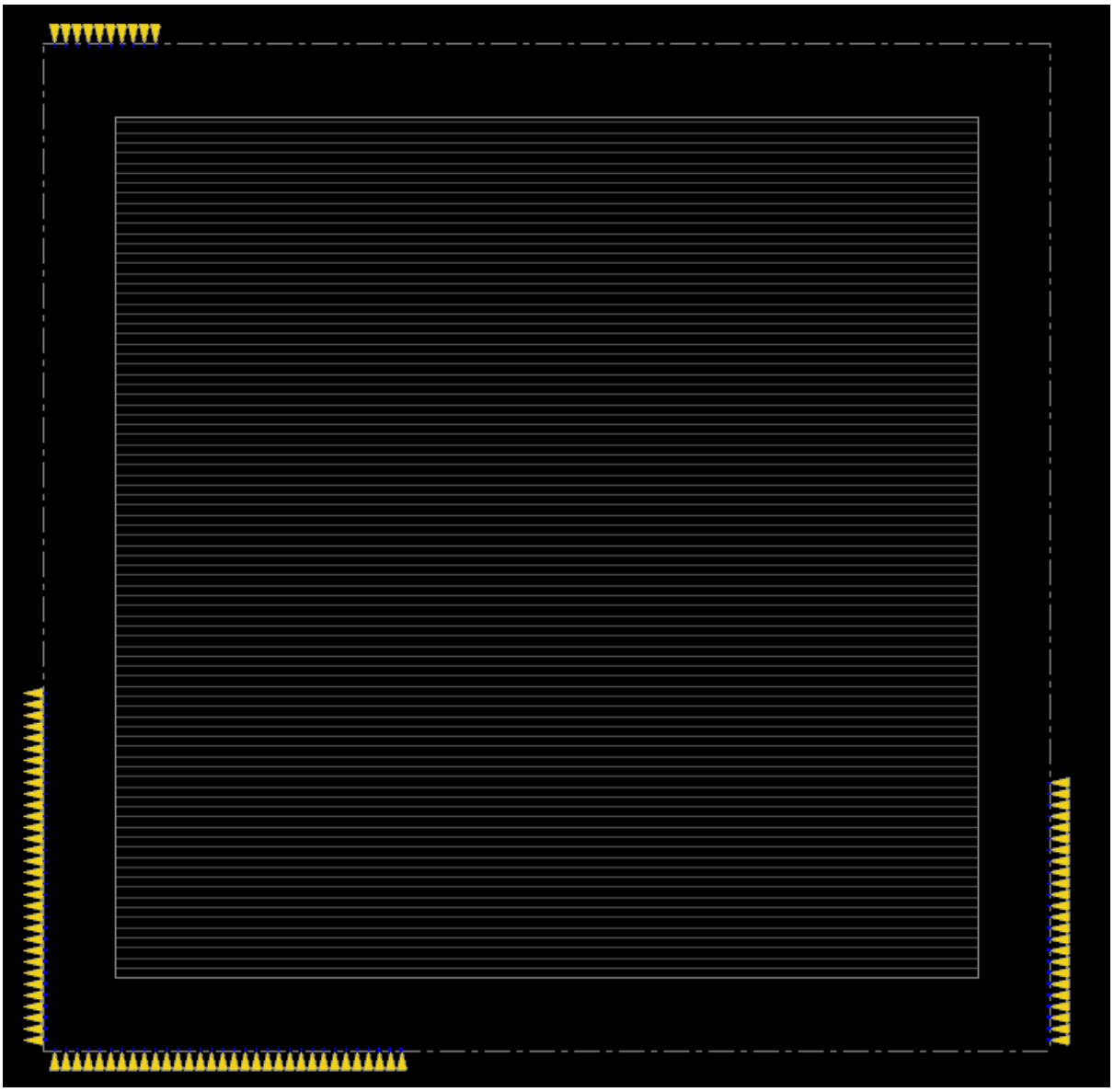
FloorPlanning

It is a critical stage in the physical design flow where you define the physical organization and layout of the entire chip before placement. It sets the blueprint for how standard cells, macros, IO pads, and power structures will be arranged on the silicon die. It defines:

- Core area
- Die area

- Macros/IPs
- IO pins
- Core utilization
- Aspect ratio





During the floorplanning stage, I calculated the core and die dimensions using the total cell area obtained from synthesis (16593.12 µm²). With a core utilization of 70%, the required core area was 23704.45 µm², resulting in a core dimension of approximately 154 µm × 154 µm. Considering a 13 µm margin on each side for IO and power rings, the final die size was set to 180 µm × 180 µm. This setup ensures sufficient routing space and optimized placement for the top design

Power Planning:

Power planning is an essential part of the ASIC physical design flow, ensuring the chip receives a stable and reliable supply of power to all standard cells, macros, memories, and I/Os.

- Ensure voltage levels remain stable throughout the chip.
- Minimize IR drop (voltage drop due to resistance in metal).

- Avoid electromigration (metal damage due to high current).
- Provide robust power to all functional blocks and corners.

PDN is the actual physical implementation of the power plan. A hierarchical network of wires that deliver power from the chip's IO pads (or bumps) to every internal standard cell and macro. **PDN Includes:**

IO pads → Core rings → Power stripes → Standard cells

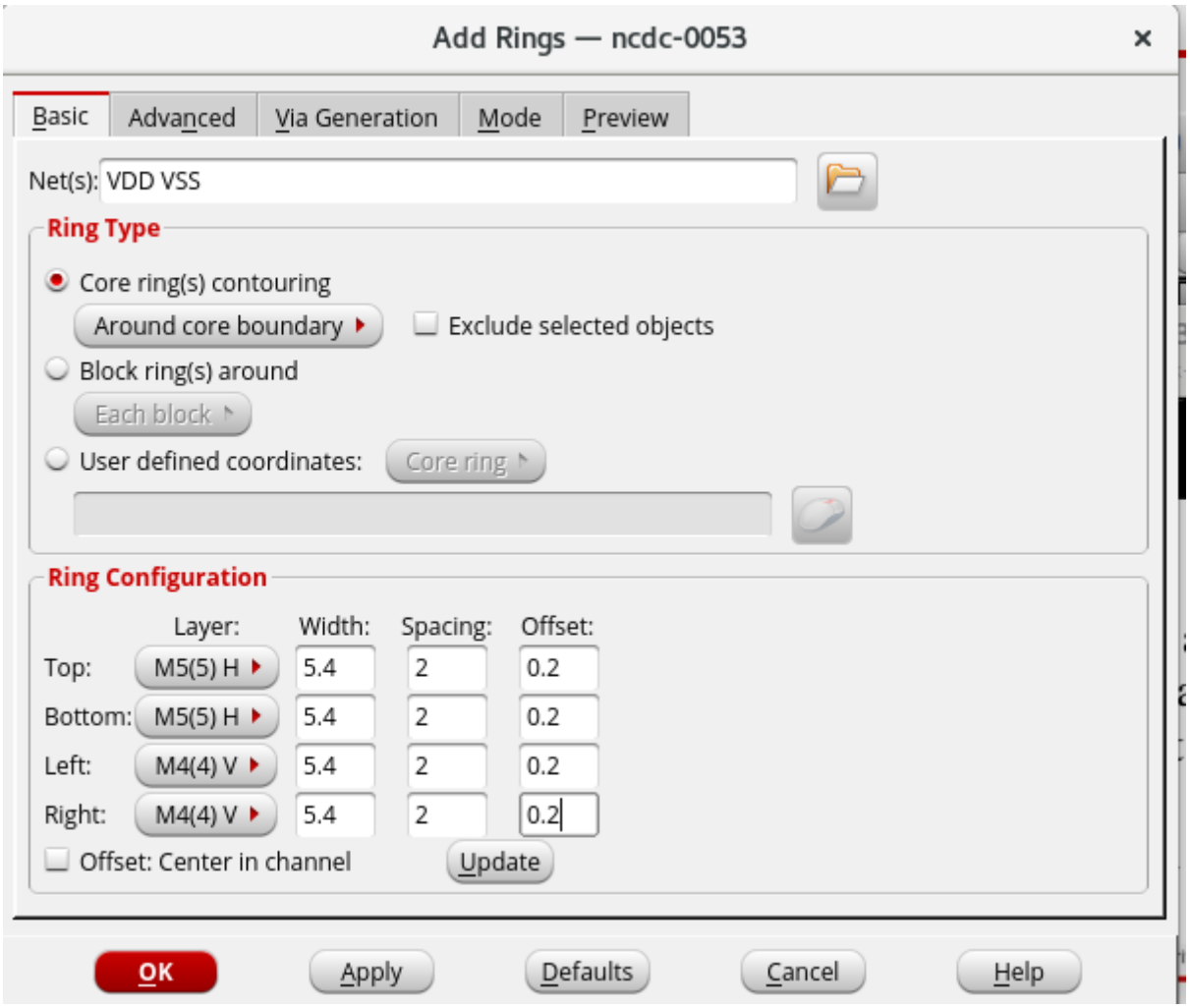
Multiple metal layers used to reduce resistance Connections to decoupling capacitors to stabilize power

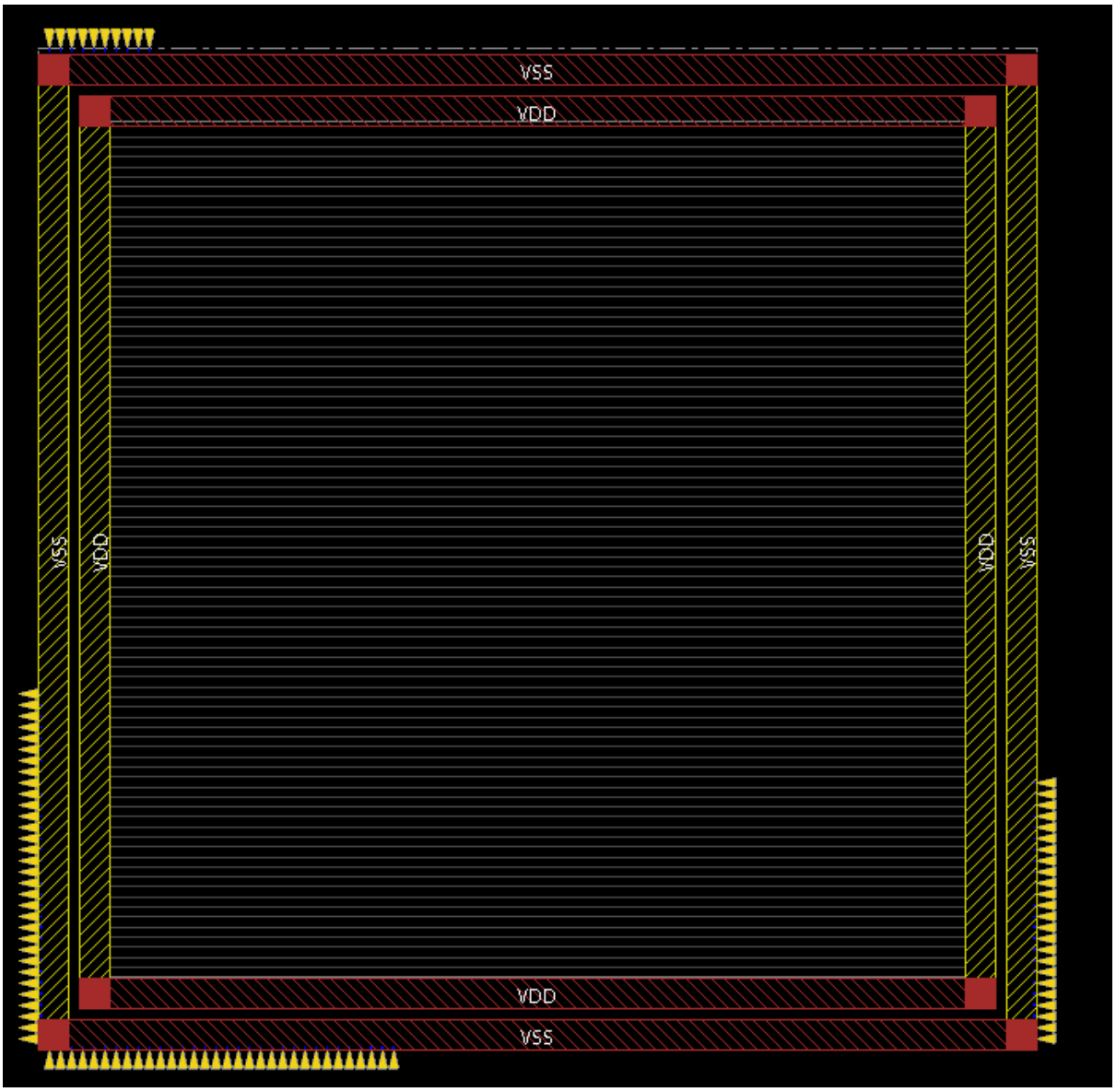
Now I have clicked on Power => Power planning => Add Ring.

Added Nets (e.g. VDD, VSS)

For the power ring higher metal layers are chosen such as metal layer 5 and 4 for wider tracks, low resistance and higher current capacity with low IR drop.

Spacing b/w rings, width and offset from boundary is set in compliance with DRC rule check given by foundry.


Add Rings — ncdc-0053
Basic
Advanced
Via Generation
Mode
Preview
Net(s): VDD VSS
Ring Type
Core ring(s) contouring
Around core boundary
Exclude selected objects
Block ring(s) around
Each block
User defined coordinates:
Core ring
Ring Configuration
Layer:
Width:
Spacing:
Offset:
Top: M5(5) H 5.4 2 0.2
Bottom: M5(5) H 5.4 2 0.2
Left: M4(4) V 5.4 2 0.2
Right: M4(4) V 5.4 2 0.2
Offset: Center in channel
Update
OK
Apply
Defaults
Cancel
Help

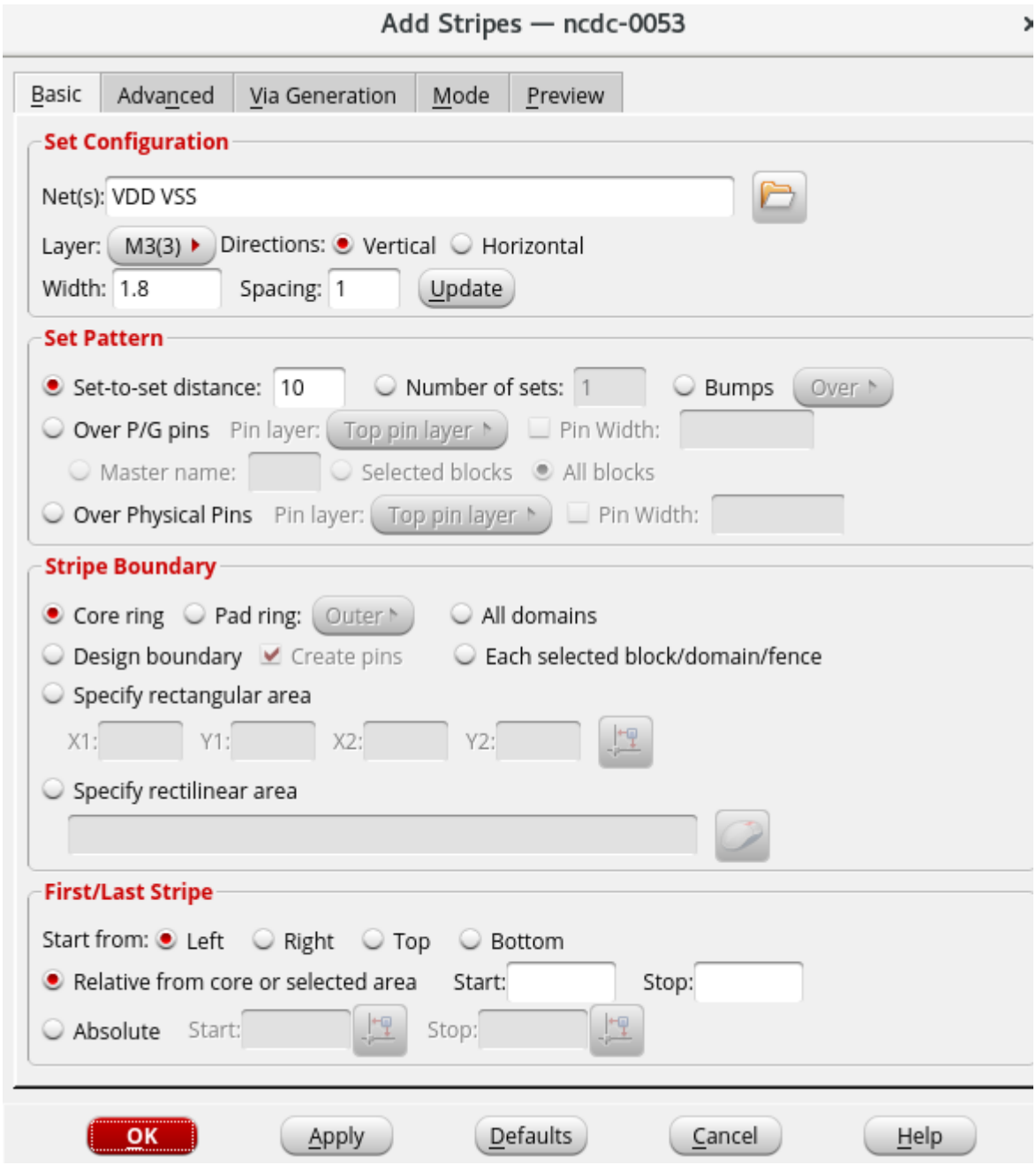


Adding Power Strips:

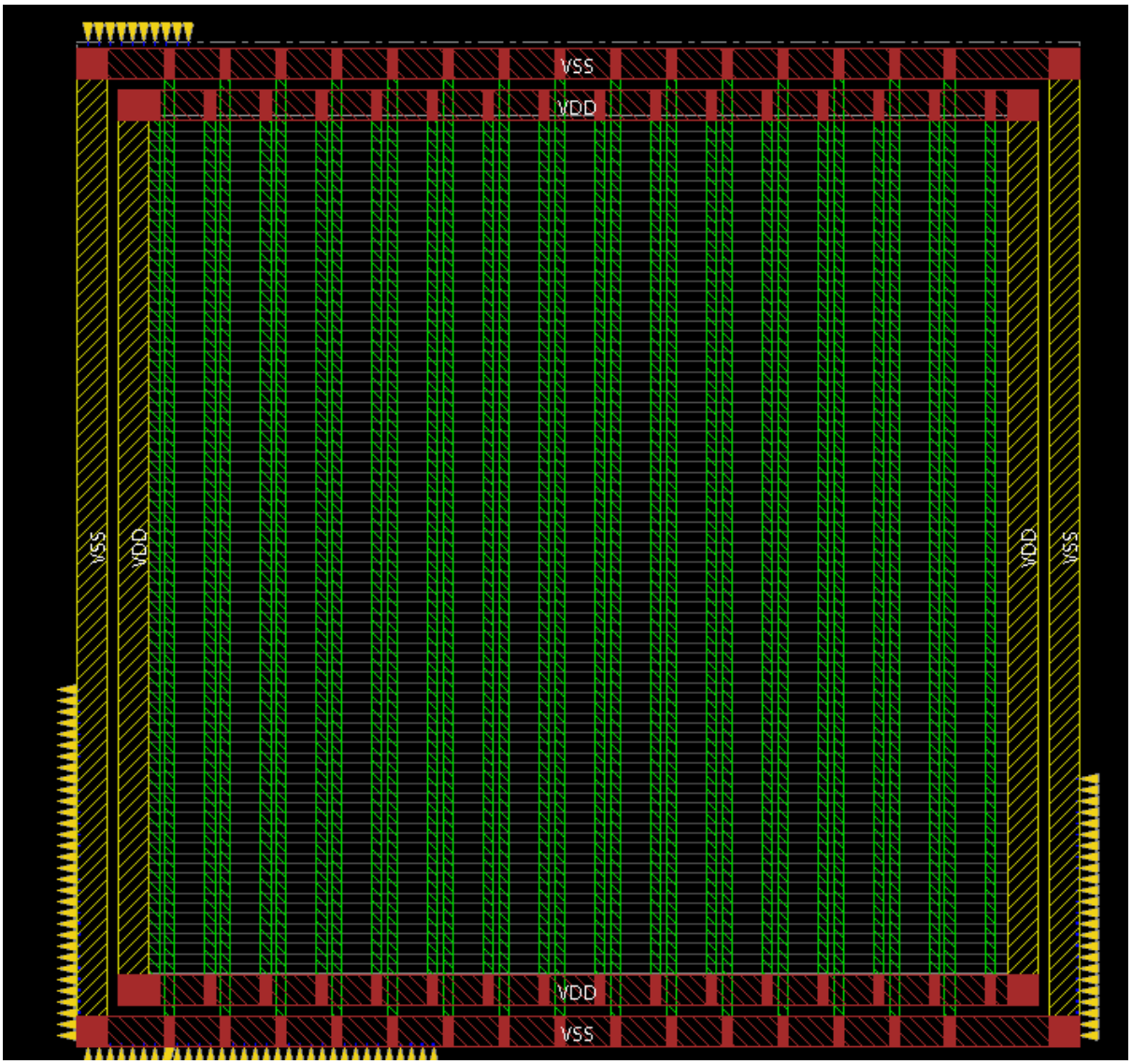
After placing a power ring around the core, metal stripes (VDD and GND) are inserted horizontally and vertically inside the core to:

- Reduce IR drop (voltage drop).
- Provide multiple power taps to standard cells.
- Support current demand evenly across the layout.

For power strips, I have clicked on power => Power planning=> Add strips



After Adding Stripes

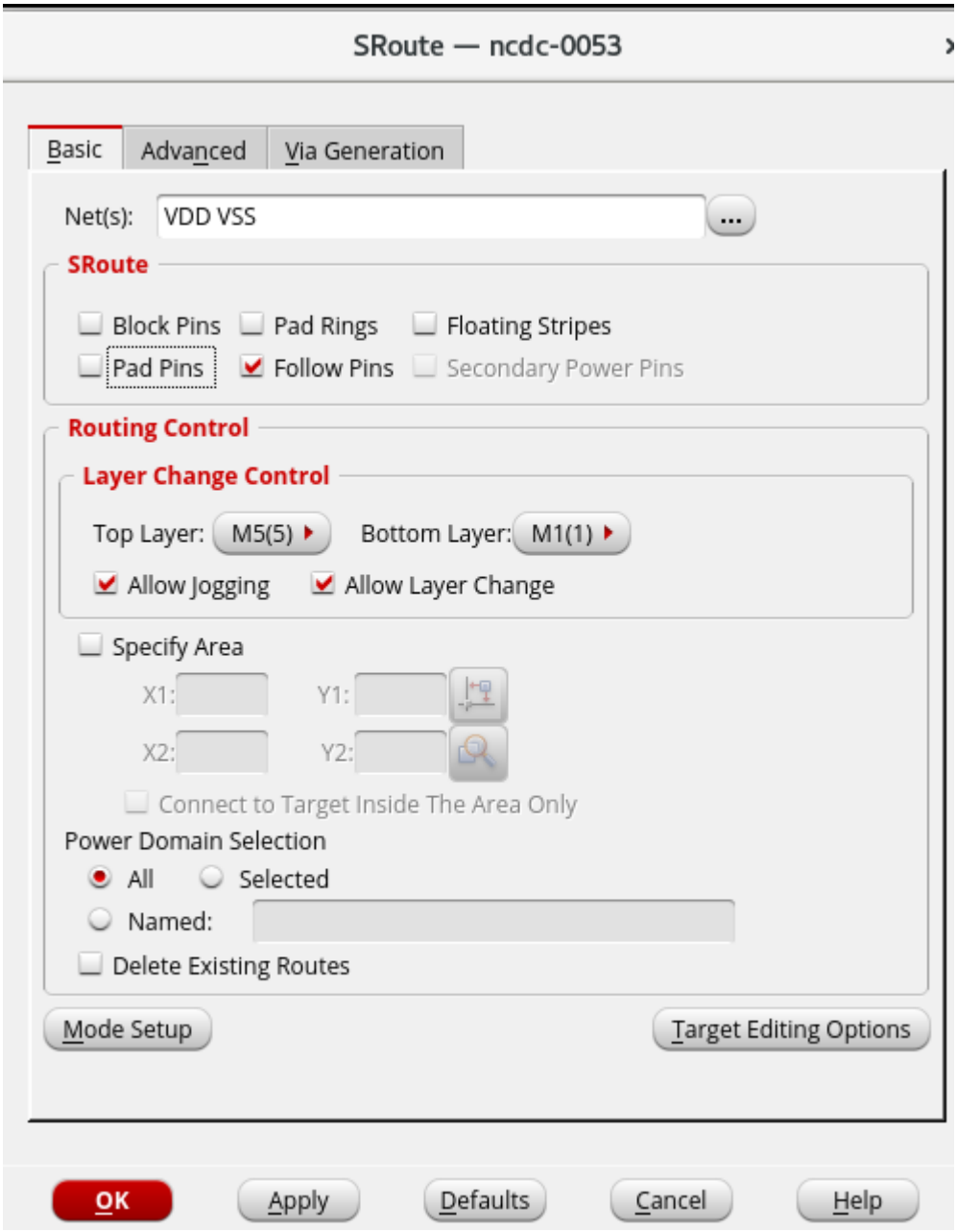


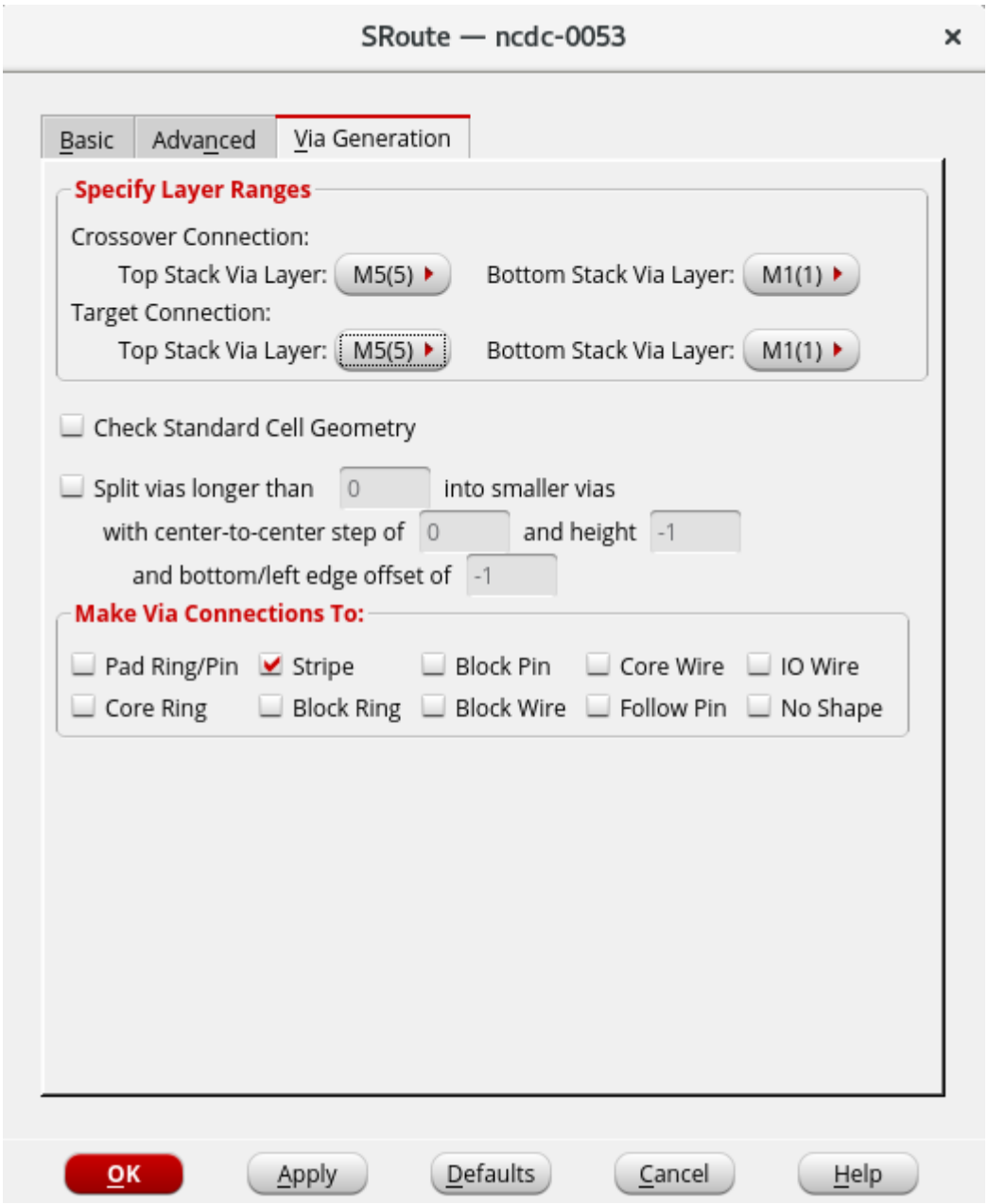
Explanation:

In the power planning stage, I added **power rings** and **power rails** to ensure a stable power distribution across the chip. The power rings for **VDD** and **VSS** were created using **metal layers M4 and M5**, as these higher layers provide wider tracks and lower resistance, minimizing IR drop. The ring surrounded the 154 µm × 154 µm core area, with a spacing of 2 µm between VDD and VSS and an overall offset of about 13 µm from the core boundary. After forming the ring, **power rails (stripes)** were added on **metal layer M3** in both horizontal and vertical directions to deliver power uniformly to the standard cells. These rails were evenly spaced (around 30 µm apart) and connected to the power ring through vias, forming a robust multi-layer power distribution network that ensures reliable voltage delivery throughout the design.

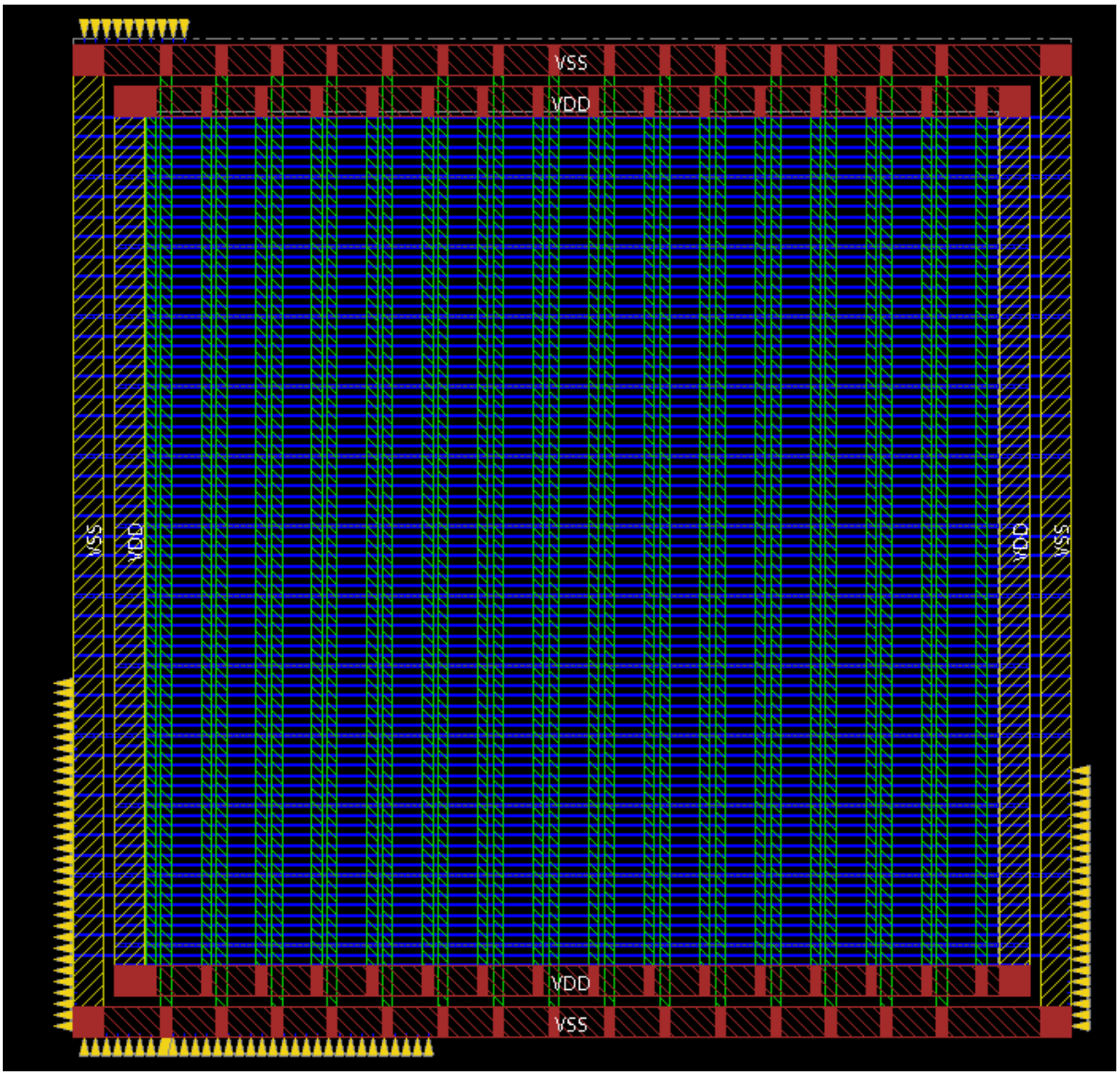
Creating power rails with special route:

I Go to Route -> Special route I used M5 for power ring, so I will select M5 as top layers while specifying power rails in my design. Vias need to be connected from power rails to power rings to get power so select same metal layer you used for power ring.





After Power Routing:



Explanation:

In the **special route (s-route)** step, I connected the power rings and power stripes to ensure continuous power delivery across all layers of the design. Since the power rings were created on **M4 and M5** and the stripes on **M3**, I used the **special routing** option to form vertical connections (vias) between these metal layers. This step ensures that all **VDD** and **VSS** nets are properly linked from the power rings to the internal rails, allowing every standard cell to receive stable power. The s-route creates low-resistance paths and minimizes IR drop by establishing a direct and reliable connection between different metal layers of the power distribution network.

Global Connections:

VDD or VSS Net

- These are the names of the global power and ground nets used throughout the design.
- They connect to the power/ground rails, stripes, and rings.

- Refers to the power/ground pins of standard cells or macros.

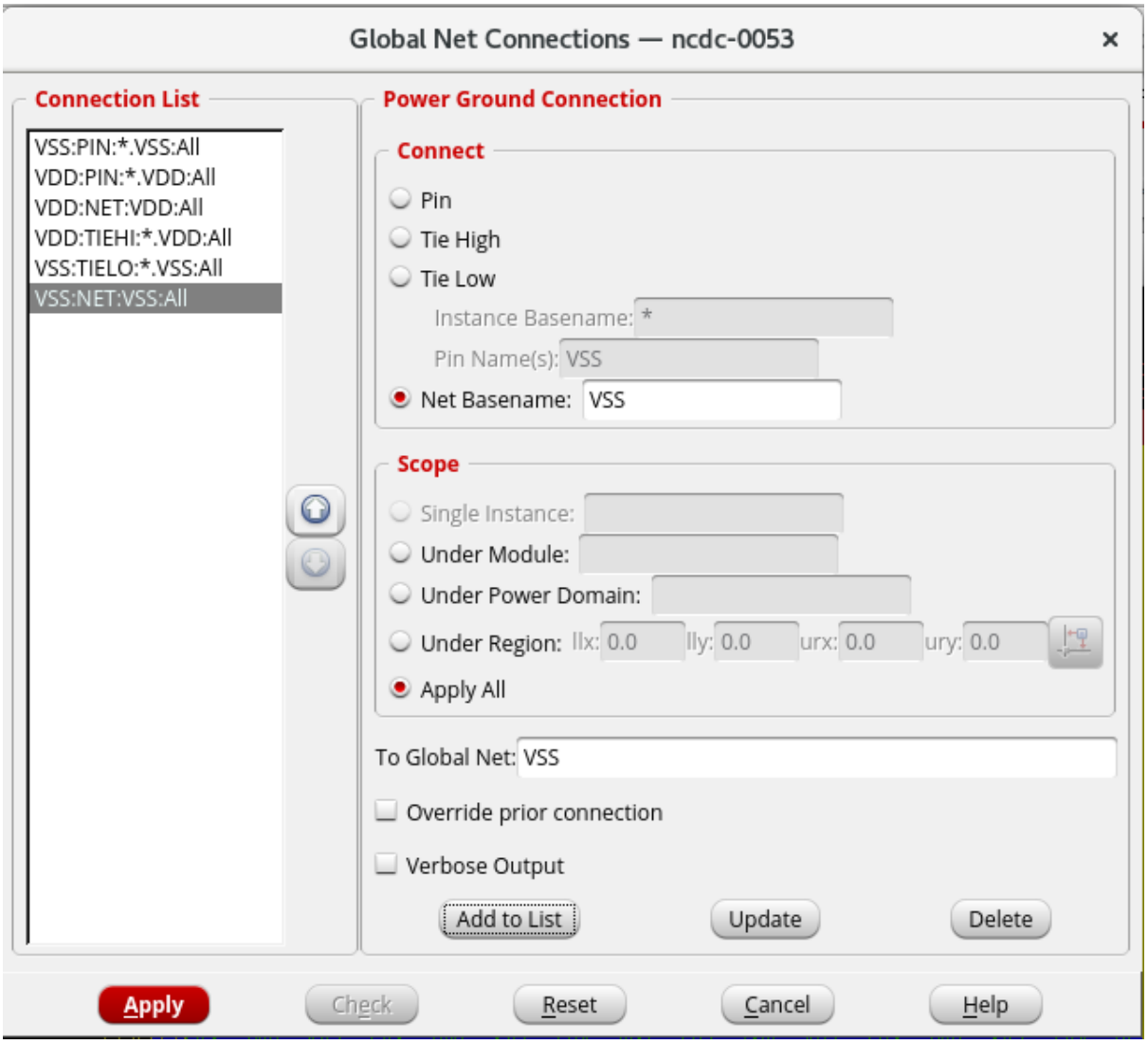
VDD TIEHI & VSS TIELO

- These refer to cells that provide a constant logic high (1) or low (0).
- Used to tie unused inputs or force constant values.

Without it, the tool may:

- Not recognize VDD or VSS connections properly
- Leave cells unpowered
- Fail LVS or IR drop checks
- Cause physical connectivity issues (like stripes/rings not actually powering logic)

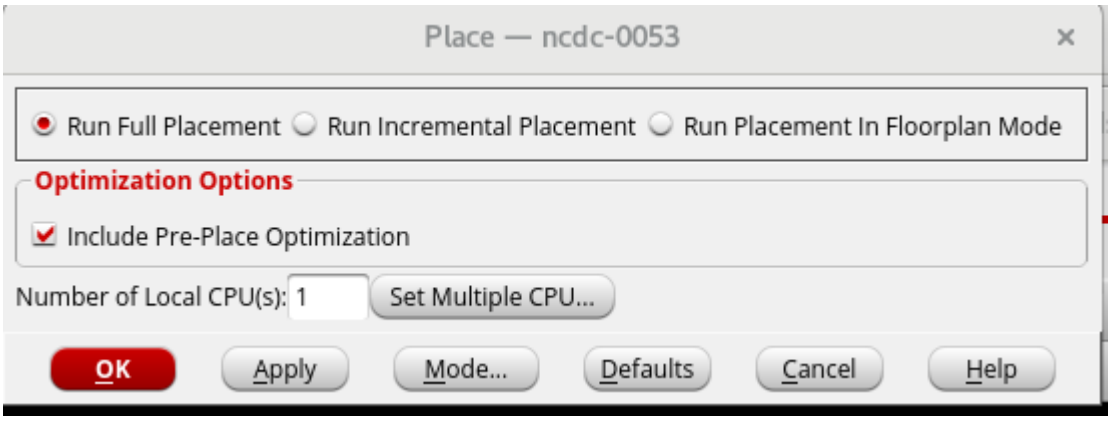
For Global connections, click on Power => Connect Global Nets.



Placement:

Placement is the stage in the physical design flow where the standard cells (logic gates) are assigned physical locations on the silicon chip, without overlapping, based on the logical netlist generated from synthesis.

We will place standard cells in our core area. These std cells are basically our RTL logic mapped to std cells from pdk.




```
Skipped repairing congestion.
[NR-eGR]              Length (um)   Vias
[NR-eGR] ---------------------------------
[NR-eGR]  M1  (1H)              0  19950
[NR-eGR]  M2  (2V)          24120  27560
[NR-eGR]  M3  (3H)          15456   9687
[NR-eGR]  M4  (4V)          22450   9511
[NR-eGR]  M5  (5H)          34303   1210
[NR-eGR]  M6  (6V)          11508    547
[NR-eGR]  M7  (7H)           8522     15
[NR-eGR]  M8  (8V)             27      2
[NR-eGR]  M9  (9H)              1      0
[NR-eGR]  AP  (10V)             0      0
[NR-eGR] ---------------------------------
[NR-eGR]      Total        116387  68482
[NR-eGR] ---------------------------------------------
---
```

Clock Tree Synthesis:

Clock Tree Synthesis (CTS) is a crucial stage in the ASIC backend flow that ensures the clock signal reaches all sequential elements (flip-flops, latches) with minimal skew and optimal delay.

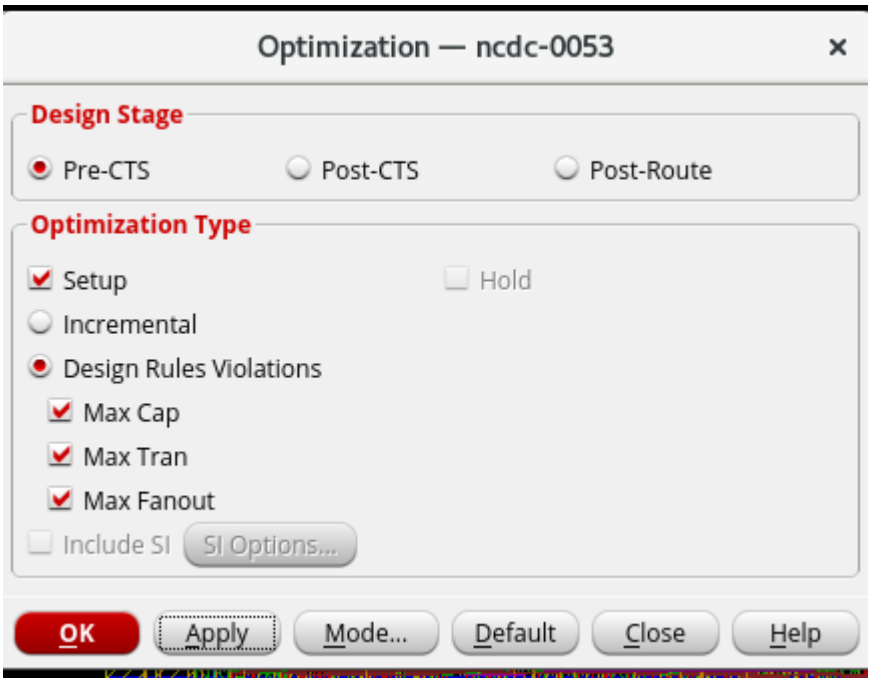
First, we will perform Pre-CTS:

Its main purpose is to prepare the design for successful CTS and post-CTS timing closure.

- Fix setup timing violations on data paths.
- Prepare for clock tree balancing
- Reduce congestion and improve cell placement
- Minimize clock skew impact
- Add buffers to improve transition, fanout and capacitance.

- Legalize placement of cells to ensure no overlaps and DRC errors.

For this I go to ECO -> Optimize design. Check Max Fanout.

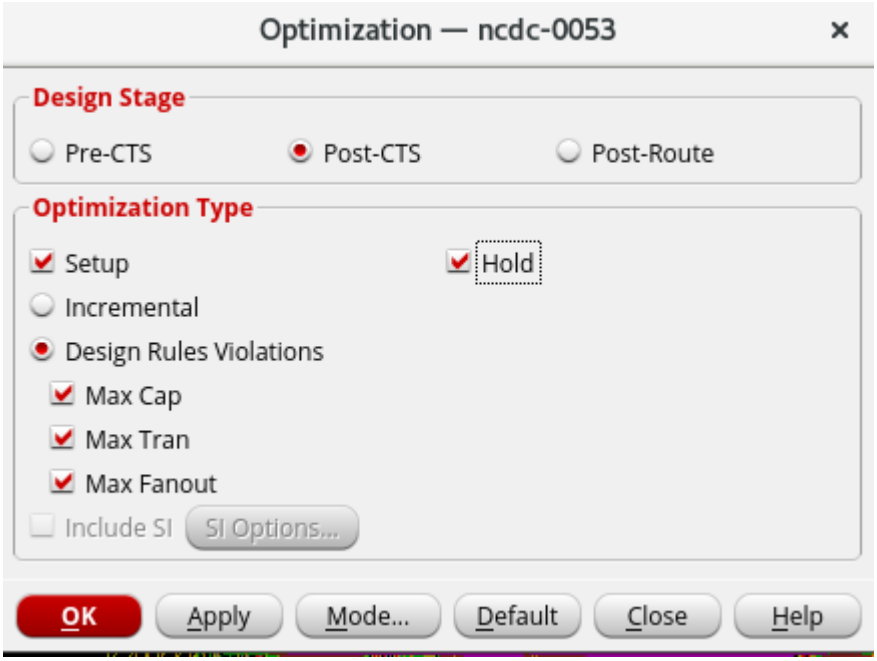



```
Setup views included:
 slow

+--------------------+---------+---------+---------+
|     Setup mode     |   all   | reg2reg | default |
+--------------------+---------+---------+---------+
|           WNS (ns):|  2.544  |  4.260  |  2.544  |
|           TNS (ns):|  0.000  |  0.000  |  0.000  |
|    Violating Paths:|    0    |    0    |    0    |
|          All Paths:|  1088   |  1024   |  1088   |
+--------------------+---------+---------+---------+

+----------------+-------------------------------+------------------+
|                |              Real             |       Total      |
|    DRVs        +------------------+------------+------------------|
|                |  Nr nets(terms)  | Worst Vio  |  Nr nets(terms)  |
+----------------+------------------+------------+------------------+
|   max_cap      |      0 (0)       |   0.000    |      0 (0)       |
|   max_tran     |      0 (0)       |   0.000    |      0 (0)       |
|   max_fanout   |      0 (0)       |     0      |      0 (0)       |
|   max_length   |      0 (0)       |     0      |      0 (0)       |
+----------------+------------------+------------+------------------+

Density: 72.070%
Routing Overflow: 0.00% H and 0.00% V
```




```
------------------------------------------------------------------
     optDesign Final Summary
------------------------------------------------------------------

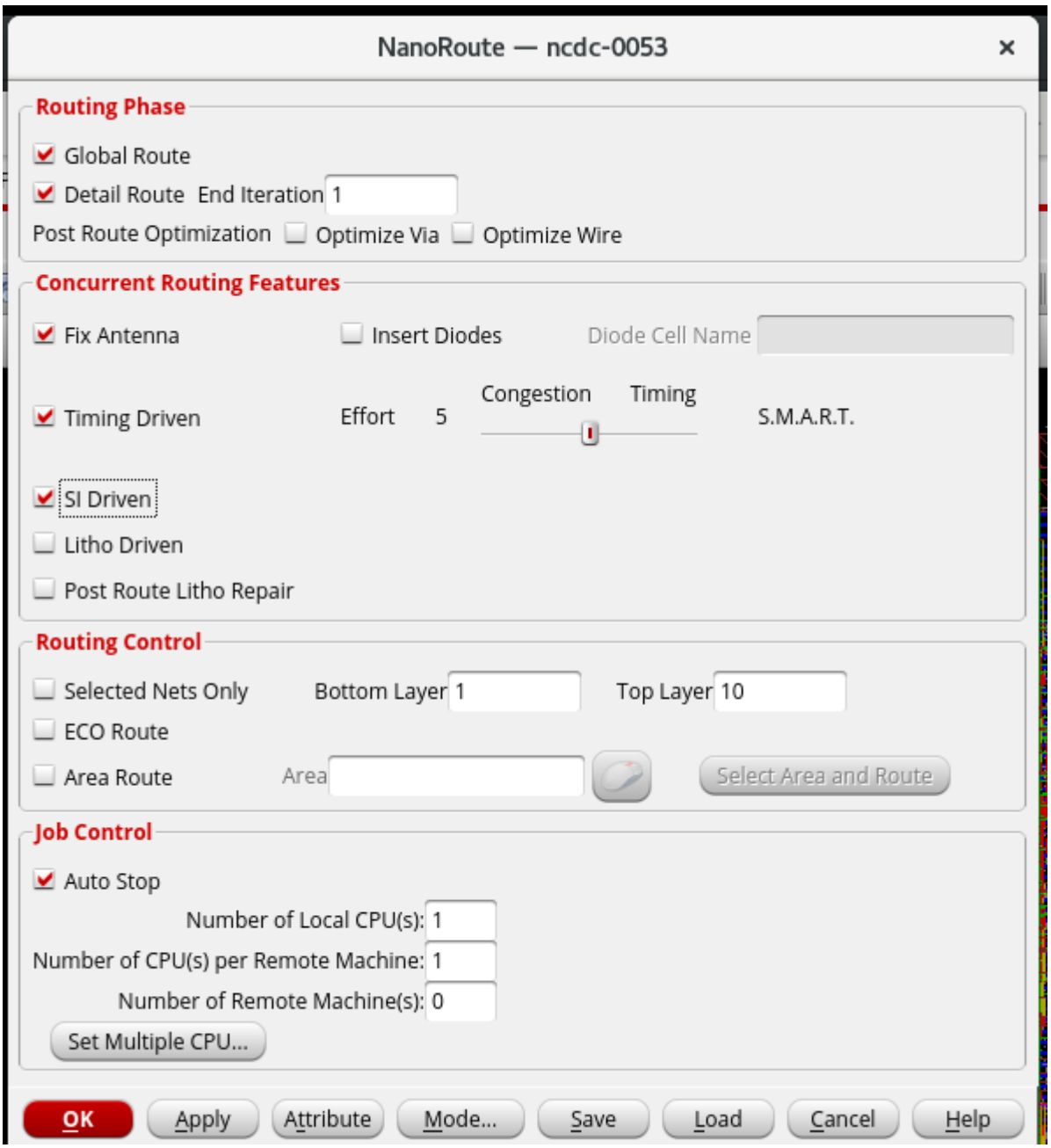
Setup views included:
 slow
Hold views included:
 fast

+--------------------+---------+---------+---------+
|     Setup mode     |   all   | reg2reg | default |
+--------------------+---------+---------+---------+
|           WNS (ns):|  2.551  |  4.286  |  2.551  |
|           TNS (ns):|  0.000  |  0.000  |  0.000  |
|    Violating Paths:|    0    |    0    |    0    |
|          All Paths:|  1088   |  1024   |  1088   |
+--------------------+---------+---------+---------+

+--------------------+---------+---------+---------+
|     Hold mode      |   all   | reg2reg | default |
+--------------------+---------+---------+---------+
|           WNS (ns):|  0.108  |  0.108  |  0.000  |
|           TNS (ns):|  0.000  |  0.000  |  0.000  |
|    Violating Paths:|    0    |    0    |    0    |
|          All Paths:|  1024   |  1024   |    0    |
+--------------------+---------+---------+---------+

+----------------+-------------------------------+------------------+
|                |              Real             |       Total      |
|    DRVs        +------------------+------------+------------------|
|                |  Nr nets(terms)  | Worst Vio  |  Nr nets(terms)  |
+----------------+------------------+------------+------------------+
|   max_cap      |      0 (0)       |   0.000    |      0 (0)       |
|   max_tran     |      0 (0)       |   0.000    |      0 (0)       |
|   max_fanout   |      0 (0)       |     0      |      0 (0)       |
|   max_length   |      0 (0)       |     0      |      0 (0)       |
+----------------+------------------+------------+------------------+

Density: 72.680%
Routing Overflow: 0.00% H and 0.00% V
```


NanoRoute — ncdc-0053
Routing Phase
Global Route
Detail Route
End Iteration 1
Post Route Optimization
Optimize Via
Optimize Wire
Concurrent Routing Features
Fix Antenna
Insert Diodes
Diode Cell Name
Timing Driven
Effort
5
Congestion
Timing
S.M.A.R.T.
SI Driven
Litho Driven
Post Route Litho Repair
Routing Control
Selected Nets Only
Bottom Layer 1
Top Layer 10
ECO Route
Area Route
Area
Select Area and Route
Job Control
Auto Stop
Number of Local CPU(s): 1
Number of CPU(s) per Remote Machine: 1
Number of Remote Machine(s): 0
Set Multiple CPU...
OK
Apply
Attribute
Mode...
Save
Load
Cancel
Help

```
#Total wire length on LAYER M4 = 25245 um.
#Total wire length on LAYER M5 = 29157 um.
#Total wire length on LAYER M6 = 18994 um.
#Total wire length on LAYER M7 = 12825 um.
#Total wire length on LAYER M8 = 0 um.
#Total wire length on LAYER M9 = 0 um.
#Total wire length on LAYER AP = 0 um.
#Total number of vias = 55984
#Total number of multi-cut vias = 15 (  0.0%)
#Total number of single cut vias = 55969 (100.0%)
#Up-Via Summary (total 55984):
#                  single-cut          multi-cut      Total
#-----------------------------------------------------------
# M1             21020 ( 99.9%)        15 (  0.1%)      21035
# M2             14053 (100.0%)         0 (  0.0%)      14053
# M3              9711 (100.0%)         0 (  0.0%)       9711
# M4              8609 (100.0%)         0 (  0.0%)       8609
# M5              1894 (100.0%)         0 (  0.0%)       1894
# M6               682 (100.0%)         0 (  0.0%)        682
#-----------------------------------------------------------
#                55969 (100.0%)        15 (  0.0%)      55984
#
#detailRoute Statistics:
```

```
*** Message Summary: 1 warning(s), 0 error(s)
```

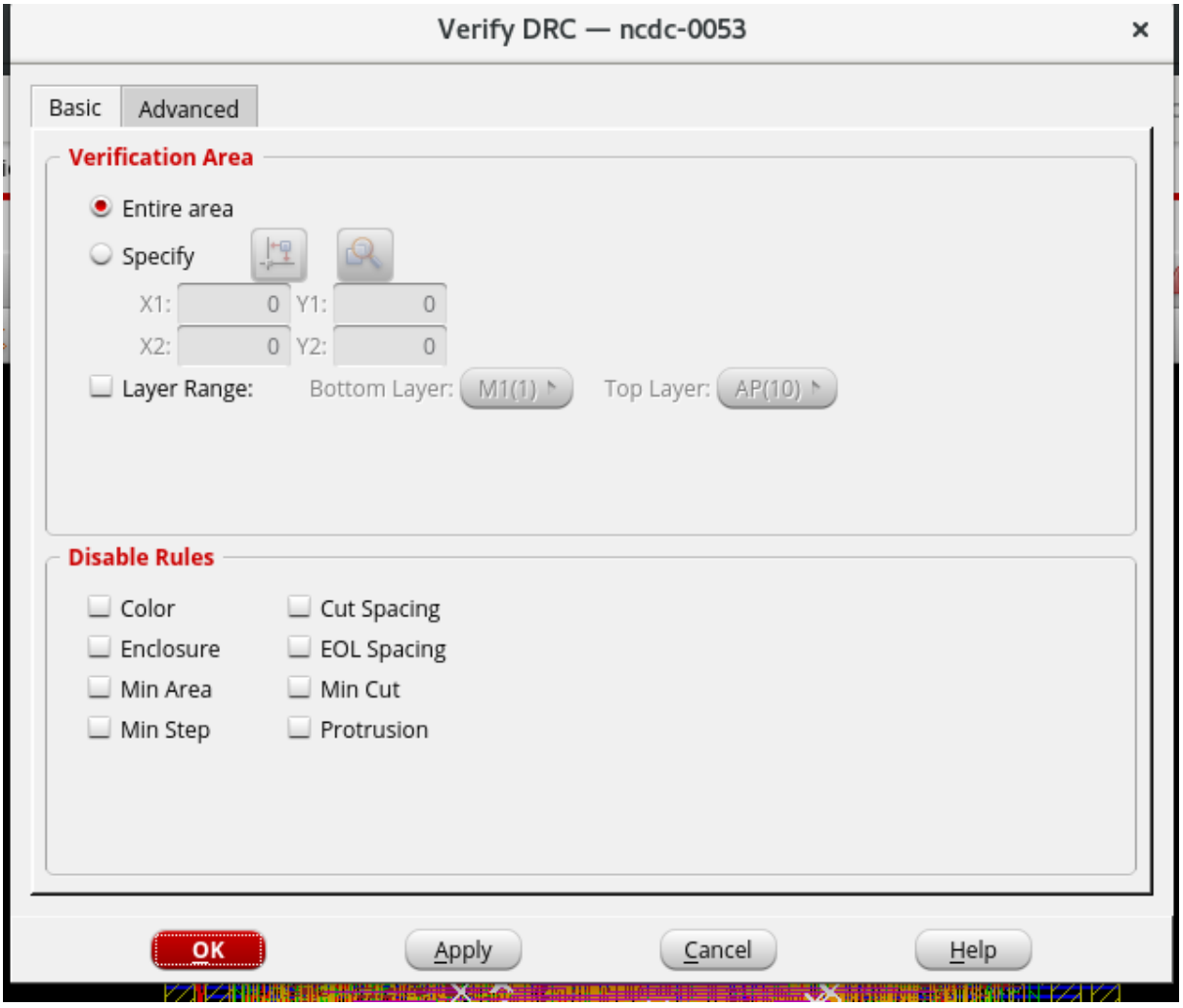
Spef file:

set_db timing_analysis_type ocv

```
Path 10: MET Setup Check with Pin pc_pc_out_reg[26]/CP
Endpoint:   pc_pc_out_reg[26]/CN (^) checked with  leading edge of 'clk'
Beginpoint: instruction[6]       (v) triggered by  leading edge of 'clk'
Path Groups: {clk}
Analysis View: slow
Other End Arrival Time          0.207
- Setup                         0.160
+ Phase Shift                  10.000
- Uncertainty                   0.010
= Required Time                10.037
- Arrival Time                  6.500
= Slack Time                    3.537
     Clock Rise Edge                     0.000
     + Input Delay                       1.000
     = Beginpoint Arrival Time           1.000
     Timing Path:
```

```
Path 10: MET Hold Check with Pin pc_pc_out_reg[0]/CP
Endpoint:   pc_pc_out_reg[0]/CN (v) checked with  leading edge of 'clk'
Beginpoint: pc_pc_out_reg[0]/Q  (v) triggered by  leading edge of 'clk'
Path Groups: {clk}
Analysis View: fast
Other End Arrival Time          0.090
+ Hold                          0.019
+ Phase Shift                   0.000
+ Uncertainty                   0.010
= Required Time                 0.119
  Arrival Time                  0.137
  Slack Time                    0.017
     Clock Rise Edge                     0.000
     + Source Insertion Delay           -0.092
     = Beginpoint Arrival Time          -0.092
     Timing Path:
```

Verify DRC:



```
 Verification Complete : 749 Viols.

*WARN: (IMPTCM-70):    Option "-droute_vio_display_type_num" for command getNanoRouteMode is obsolete an
 has been replaced by "-drouteVioDisplayTypeNum". The obsolete option still works in this release but to
avoid this warning and to ensure compatibility with future releases, update your script to use "-drouteV
oDisplayTypeNum".
Violation Summary By Layer and Type:

                Short   MetSpc   EOLSpc   CutSpc      Mar   CShort   Totals
      M1          409      282       11        0        0        0      702
      VIA1          0        0        0        9        0        1       10
      M2           20        8        7        0        2        0       37
      Totals      429      290       18        9        2        1      749

*** End Verify DRC (CPU: 0:00:00.8  ELAPSED TIME: 1.00  MEM: 0.0M) ***
```

During DRC verification I found 749 violations concentrated on lower metal layers (M1/M2), dominated by M1 shorts and metal spacing issues. I triaged the errors by first visualizing violation hot-spots, then fixing shorts (rerouting overlapping nets and correcting misplaced metal), and next addressing spacing and end-of-line violations by rerouting, adjusting stripe offsets, and relocating congested signals to higher metal layers. After incremental rip-up-and-reroute with increased routing effort and careful via redistribution for power connections, I re-ran DRC to verify fixes and iterated until violations were resolved.

```
Begin Summary
  Found no problems or warnings.
End Summary

End Time: Sat Oct 11 18:28:59 2025
Time Elapsed: 0:00:00.0

******** End: VERIFY CONNECTIVITY ********
  Verification Complete : 0 Viols.  0 Wrngs.
  (CPU Time: 0:00:00.1  MEM: 0.000M)
```

Critical Analysis:

In this lab, I designed and integrated different modules to form a complete single-cycle processor. Each module including the Program Counter, Instruction Memory, Control Unit, Register File, and ALU was developed and verified individually before integration. The design successfully executed basic instructions within one clock cycle, demonstrating the fundamental working of a processor datapath. During implementation, the main challenge was ensuring proper signal connectivity and timing synchronization between modules, especially while integrating the instruction memory and control unit. Debugging the control signals and verifying instruction decoding required careful observation of simulation waveforms. Overall, the lab provided a clear understanding of how data flows through the processor and how control signals coordinate the execution process. However, the single-cycle design is not efficient for complex instructions, as all operations complete in one long cycle. Future improvement could involve implementing a pipelined design to enhance performance and clock efficiency. I have performed complete front end and back end of single cycle processor.